

Part 6

অ্যানোমালি ডিটেকশন (Anomaly Detection)

বাস্তব জগতের প্রায় প্রতিটি ডেটাসেটে এমন কিছু পর্যবেক্ষণ (Observation) থাকে যা বাকি সব ডেটার সাধারণ প্যাটার্ন থেকে উল্লেখযোগ্যভাবে ভিন্ন। এই ভিন্ন বা ব্যতিক্রমী পর্যবেক্ষণগুলোকে বলা হয় Anomaly বা Outlier, এবং সেগুলো স্বয়ংক্রিয়ভাবে শনাক্ত করার প্রক্রিয়াকে বলা হয় Anomaly Detection। এই ধারণাটি পরিসংখ্যানের জগতে বহু পুরনো — বিংশ শতাব্দীর মাঝামাঝি সময়ে শিল্প উৎপাদনে Statistical Process Control এবং Grubbs' Test-এর মতো সাধারণ পরিসংখ্যানিক পদ্ধতি দিয়ে এই কাজ শুরু হয়েছিল, যেখানে একটিমাত্র ফিচারের ওপর ভিত্তি করে (যেমন গড় থেকে কয়টি স্ট্যান্ডার্ড ডিভিয়েশন দূরে একটি মান অবস্থিত) সিদ্ধান্ত নেওয়া হতো। কিন্তু আধুনিক ডেটাসেট সাধারণত বহু-মাত্রিক (High-Dimensional) এবং জটিল, তাই এই Part-এ আলোচিত দুটি অ্যালগরিদম — Isolation Forest এবং One-Class SVM — এমন দুটি সম্পূর্ণ ভিন্ন দার্শনিক দৃষ্টিভঙ্গি থেকে এই সমস্যার সমাধান করে যা আধুনিক Machine Learning-ভিত্তিক Anomaly Detection-এর ভিত্তি গঠন করেছে।

Anomaly Detection-কে সাধারণত তিনটি প্রকারে ভাগ করা হয়: Point Anomaly (একটি একক পয়েন্ট বাকি ডেটা থেকে ভিন্ন, যেমন একটি অস্বাভাবিক বড় ব্যাংক লেনদেন), Contextual Anomaly (একটি নির্দিষ্ট প্রেক্ষাপটে অস্বাভাবিক, যেমন গ্রীষ্মকালে হঠাৎ তুষারপাতের তাপমাত্রা), এবং Collective Anomaly (এককভাবে স্বাভাবিক হলেও একত্রে অস্বাভাবিক একটি প্যাটার্ন তৈরি করে, যেমন সার্ভার লগে ধারাবাহিক অস্বাভাবিক অ্যাক্সেস প্যাটার্ন)। এই Part-এ আলোচিত দুটি অ্যালগরিদম মূলত Point Anomaly এবং Novelty Detection সমস্যার সমাধানে কেন্দ্রীভূত, এবং উভয়ই Unsupervised বা Semi-Supervised পদ্ধতিতে কাজ করে — অর্থাৎ Anomaly-এর কোনো Labeled উদাহরণ ছাড়াই এগুলো ব্যতিক্রমী পয়েন্ট শনাক্ত করতে সক্ষম, যা বাস্তব জগতে অত্যন্ত গুরুত্বপূর্ণ, কারণ Anomaly বা Fraud-এর Labeled ডেটা সাধারণত পাওয়া যায় না বা সংখ্যায় অত্যন্ত কম থাকে।

এই Part-এ যা থাকছে: Chapter 22-এ আলোচিত হয়েছে Isolation Forest — একটি Tree-ভিত্তিক, Random Partitioning-নির্ভর দ্রুতগতির অ্যালগরিদম। Chapter 23-এ আলোচিত হয়েছে One-Class SVM — একটি Kernel-ভিত্তিক, জ্যামিতিক সীমানা (Boundary) নির্মাণকারী অ্যালগরিদম। দুই অধ্যায়ের শেষে একটি বিস্তারিত তুলনামূলক বিশ্লেষণ এবং একটি সিদ্ধান্ত-গ্রহণ ফ্লোচার্ট দেওয়া হয়েছে, যা বাস্তব প্রজেক্টে সঠিক অ্যালগরিদম বেছে নিতে সাহায্য করবে।

Chapter 22 — Isolation Forest

২২.১ পরিচিতি ও দর্শন (Introduction and Philosophy)

Isolation Forest হলো একটি Unsupervised, Tree-ভিত্তিক Anomaly বা Outlier Detection Algorithm। এটি ২০০৮ সালে Fei Tony Liu, Kai Ming Ting এবং Zhi-Hua Zhou কর্তৃক প্রস্তাবিত হয় এবং প্রকাশিত হওয়ার পর থেকেই এটি তার অস্বাভাবিক গতি ও সরলতার কারণে শিল্পক্ষেত্রে ব্যাপকভাবে গৃহীত হয়েছে। অন্যান্য বেশিরভাগ Outlier Detection পদ্ধতি প্রথমে ডেটার মধ্যে 'স্বাভাবিক' বা Normal অংশের একটি বিস্তারিত প্রোফাইল তৈরি করে (যেমন Density বা Distance পরিমাপ করে), তারপর যা সেই প্রোফাইলের সাথে না মেলে তাকে Outlier হিসেবে চিহ্নিত করে। Isolation Forest সম্পূর্ণ ভিন্ন এবং অভিনব একটি দর্শনে কাজ করে — এটি সরাসরি ধরে নেয় যে, Anomaly বা ব্যতিক্রমী পয়েন্টগুলো সংখ্যায় কম (Few) এবং অন্যান্য স্বাভাবিক ডেটা থেকে ভিন্ন (Different), তাই তাদের 'বিচ্ছিন্ন' (Isolate) করে ফেলা সাধারণ পয়েন্টের তুলনায় অনেক সহজ ও দ্রুত।

এই দৃষ্টিভঙ্গির একটি গুরুত্বপূর্ণ শিল্পগত তাৎপর্য আছে: Density-ভিত্তিক বা Distance-ভিত্তিক পদ্ধতিগুলো (যেমন Local Outlier Factor বা One-Class SVM) সাধারণত প্রতিটি পয়েন্টের জন্য অন্য সব পয়েন্টের সাথে দূরত্ব বা ঘনত্ব গণনা করতে হয়, যা বড় ডেটাসেটে গণনাগতভাবে ব্যয়বহুল (সাধারণত $O(n^2)$ বা তার বেশি জটিলতা)। Isolation Forest এই সমস্যা সম্পূর্ণভাবে এড়িয়ে যায়, কারণ এটি কোনো দূরত্ব বা ঘনত্ব গণনা করে না — এটি শুধু Random ভাবে ডেটাকে ভাগ করে। এই কারণে এর গড় Time Complexity প্রায় Linear, $O(n \log n)$, যা একে বৃহৎ, বাস্তব-জগতের শিল্প ডেটাসেটে (যেমন কোটি কোটি ব্যাংক লেনদেন) ব্যবহারের জন্য বিশেষভাবে উপযোগী করে তোলে।

২২.২ উদাহরণ (Intuition)

কল্পনা করুন, একটি ঘরে ১০০ জন সাধারণ উচ্চতার (৫ থেকে ৫.৮ ফুট) মানুষ এবং মাত্র ১ জন অস্বাভাবিক লম্বা (৭ ফুট) মানুষ আছেন। যদি আপনি এলোমেলোভাবে প্রশ্ন করে (যেমন: 'উচ্চতা কি ৬ ফুটের বেশি?') মানুষদের ভাগ করতে থাকেন, তাহলে ওই অস্বাভাবিক লম্বা মানুষটি মাত্র ১টি প্রশ্নেই বাকি সবার থেকে আলাদা হয়ে যাবে, কারণ সে সংখ্যাগরিষ্ঠ থেকে অনেক দূরে অবস্থান করছে এবং তাকে আলাদা করার মতো র্যান্ডম প্রশ্নের সংখ্যা অনেক বেশি। কিন্তু সাধারণ উচ্চতার মানুষদের একে অপরের থেকে সম্পূর্ণ আলাদা করতে আরও অনেক বেশি প্রশ্নের প্রয়োজন হবে, কারণ তারা একে অপরের এত কাছাকাছি যে তাদের আলাদা করতে বহু ধাপে ভাগ করতে হয়। Isolation Forest ঠিক এই নীতিতেই কাজ করে — যে পয়েন্ট যত কম প্রশ্নে (Split) আলাদা হয়ে যায়, তাকে তত বেশি Anomalous ধরা হয়।

সাধারণ পয়েন্ট (ঘন এলাকা)	Anomaly পয়েন্ট (বিরল, দূরবর্তী)
<pre>x x x x x x x x x x x x x x</pre>	<pre> A <-- অনেক দূরে বিচ্ছিন্ন</pre>
ঘন এলাকা বিচ্ছিন্ন করতে বহু Split প্রয়োজন => বড় Path Length => Normal	A কে বিচ্ছিন্ন করতে মাত্র ১-২টি Split যথেষ্ট => ছোট Path Length => Anomaly

২২.৩ মূল কার্যপ্রণালী (How it Works) — ধাপে ধাপে

১. Random Partitioning

বহুসংখ্যক Isolation Tree (iTree) তৈরি করা হয়। প্রতিটি গাছ তৈরির সময়, প্রতিটি Node-এ একটি র্যান্ডম ফিচার বেছে নেওয়া হয় এবং সেই ফিচারের Minimum ও Maximum মানের মধ্যে একটি সম্পূর্ণ র্যান্ডম Split Value বেছে নিয়ে ডেটাকে দুই ভাগে ভাগ করা হয়। এখানে লক্ষণীয় যে, সাধারণ Decision Tree-এর মতো Gini Impurity বা Entropy গণনা করে 'সেরা' Split খোঁজা হয় না — সম্পূর্ণভাবে Random-ভাবে ভাগ করা হয়, যা এই অ্যালগরিদমকে অত্যন্ত দ্রুতগতির করে তোলে।

২. পুনরাবৃত্তিমূলক বিভাজন

এই র্যান্ডম Split প্রতিটি নতুন Sub-node-এ আবার পুনরাবৃত্তি হতে থাকে, যতক্ষণ না প্রতিটি ডেটা পয়েন্ট সম্পূর্ণভাবে একটি একক Leaf-এ বিচ্ছিন্ন (Isolated) হয়ে যায়, অথবা গাছটি একটি পূর্বনির্ধারিত সর্বোচ্চ গভীরতায় পৌঁছে যায়।

৩. Path Length পরিমাপ

প্রতিটি ডেটা পয়েন্টের জন্য গণনা করা হয়, Root Node থেকে শুরু করে সেই পয়েন্ট সম্পূর্ণভাবে বিচ্ছিন্ন (Isolate) হতে ঠিক কতগুলো Split বা কতটুকু Path Length প্রয়োজন হলো — একে বলা হয় $h(x)$ ।

৪. Ensemble Averaging

একটি মাত্র গাছের ফলাফলের ওপর নির্ভর করা নির্ভরযোগ্য নয়, কারণ একটি নির্দিষ্ট Random Split-এর কারণে ফলাফল পরিবর্তিত হতে পারে। তাই এই সম্পূর্ণ প্রক্রিয়া বহুসংখ্যক গাছ ($n_estimators$, সাধারণত ১০০টি) জুড়ে পুনরাবৃত্তি হয় এবং প্রতিটি পয়েন্টের সবগুলো গাছ জুড়ে গড় Path Length $E(h(x))$ বের করা হয়।

৫. Anomaly Score নির্ধারণ

যেসব পয়েন্টের গড় Path Length তুলনামূলকভাবে ছোট (অর্থাৎ তারা দ্রুত বিচ্ছিন্ন হয়ে যায়), তাদের Anomaly হিসেবে চিহ্নিত করা হয়। বিপরীতভাবে, যেসব পয়েন্টের Path Length বড় (বিচ্ছিন্ন হতে অনেক বেশি Split প্রয়োজন হয়), তাদের স্বাভাবিক বা Normal বলে গণ্য করা হয়, কারণ তারা অন্যান্য অনেক পয়েন্টের সাথে মিলেমিশে থাকে।

iTree নির্মাণ প্রবাহ (Pipeline):

```
ডেটাসেট -> [Random Feature বেছে নাও] -> [Random Split Value বেছে নাও]
        -> [বামে/ডানে ভাগ করো] -> পুনরাবৃত্তি (প্রতিটি Sub-node-এ)
        -> Leaf-এ পৌঁছানো পর্যন্ত ->  $h(x)$  রেকর্ড করো
```

```
 $n\_estimators$  সংখ্যক গাছ জুড়ে গড় করো ->  $E(h(x))$  ->  $s(x, n)$  -> Anomaly Score
```

২২.৪ সংখ্যাসূচক উদাহরণ — একটি ছোট Isolation Tree তৈরি হওয়ার প্রক্রিয়া

ধরা যাক, আমাদের কাছে একটি 1D ফিচার (Transaction Amount) এর ছয়টি মান আছে: 10, 12, 11, 13, 9, 500। এখানে 500 একটি স্পষ্ট Anomaly (হয়তো একটি জালিয়াতি লেনদেন), বাকি পাঁচটি মান স্বাভাবিক লেনদেনের পরিসরে (৯ থেকে ১৩) রয়েছে।

Split ধাপ	র‍্যান্ডম Threshold	ফলাফল
Split ১ (Root)	Amount > 250	বামে: {10,12,11,13,9} (৫টি পয়েন্ট) — ডানে: {500} (মাত্র ১টি পয়েন্ট, তাৎক্ষণিকভাবে বিচ্ছিন্ন!)
Split ২ (বাম শাখায়)	Amount > 11.5	বামে: {10,11,9} — ডানে: {12,13}
Split ৩ (আরও ভাঙতে থাকবে)	...	প্রতিটি স্বাভাবিক পয়েন্ট সম্পূর্ণ বিচ্ছিন্ন হতে আরও ২-৩টি Split প্রয়োজন হবে

লক্ষ করুন, 500 মানটি মাত্র ১টি Split-এই (Path Length $h(x) = 1$) সম্পূর্ণভাবে বাকি সবার থেকে বিচ্ছিন্ন হয়ে গেছে, কারণ এটি বাকি ডেটার তুলনায় সংখ্যাগতভাবে অনেক দূরে অবস্থিত। অন্যদিকে, 10, 11, 12, 13, 9 — এই মানগুলো একে অপরের অত্যন্ত কাছাকাছি থাকায় তাদের সম্পূর্ণ আলাদা করতে অনেক বেশি Split (Path Length ৩ থেকে ৪ বা তার বেশি) প্রয়োজন হবে। এই সংক্ষিপ্ত Path Length-ই 500-কে একটি স্পষ্ট Anomaly হিসেবে চিহ্নিত করে দেয়।

সম্পূর্ণ ম্যানুয়াল গাণিতিক ওয়াকথ্রু (Full Manual Walkthrough)

উপরের উদাহরণে $n = 6$ টি পয়েন্ট আছে। ধরা যাক একটি একক গাছে 500-এর Path Length $h(500) = 1$, এবং বাকি পাঁচটি স্বাভাবিক পয়েন্টের গড় Path Length প্রায় $h \approx 3.5$ (যেহেতু তাদের বিচ্ছিন্ন হতে গড়ে ৩ থেকে ৪টি Split প্রয়োজন)। প্রথমে $c(n)$ গণনা করা যাক, যেখানে $n = 6$:

$$H(5) = 1 + 1/2 + 1/3 + 1/4 + 1/5 \approx 2.2833$$

$$c(6) = 2 \times H(5) - (2 \times 5)/6 = 2 \times 2.2833 - 1.6667 \approx 2.90$$

এবার 500 পয়েন্টের জন্য Anomaly Score:

$$s(500, 6) = 2^{(-1/2.90)} = 2^{(-0.345)} \approx 0.788$$

এবং একটি স্বাভাবিক পয়েন্ট (যেমন 11)-এর জন্য, যদি তার গড় Path Length $h \approx 3.5$ হয়:

$$s(11, 6) = 2^{(-3.5/2.90)} = 2^{(-1.207)} \approx 0.433$$

পর্যবেক্ষণ: লক্ষ করুন, 500-এর Score (≈ 0.79) 1-এর কাছাকাছি এবং স্পষ্টভাবে 0.5-এর অনেক ওপরে, যা তাকে একটি সম্ভাব্য Anomaly হিসেবে চিহ্নিত করে। অন্যদিকে, 11-এর Score (≈ 0.43) 0.5-এর নিচে, যা নিশ্চিত করে যে এটি ডেটার একটি স্বাভাবিক ও ঘন অংশের সদস্য। এই ম্যানুয়াল হিসাব যেকোনো ক্যালকুলেটরে যাচাই করা সম্ভব এবং এটি দেখায় কীভাবে একটি বিমূর্ত সূত্র থেকে একটি বাস্তব, ব্যাখ্যাযোগ্য সিদ্ধান্তে পৌঁছানো যায়।

২২.৫ গাণিতিক ভিত্তি — Anomaly Score

একটি পয়েন্ট x -এর Anomaly Score $s(x, n)$ নিচের সূত্রে গণনা করা হয়, যেখানে $E(h(x))$ হলো সবগুলো গাছ জুড়ে x -এর গড় Path Length, এবং $c(n)$ হলো n সংখ্যক Sample-এর একটি সাধারণ Binary Search Tree-তে গড় Path Length-এর একটি Normalization ফ্যাক্টর, যা Score-কে ০ থেকে ১-এর মধ্যে আনতে ব্যবহৃত হয়:

$$s(x, n) = 2^{-(E(h(x)) / c(n))}$$

এখানে $c(n)$ নিচের সূত্রে গণনা করা হয় $H(i)$ হলো Harmonic Number, যা প্রায় $\ln(i) + 0.5772$ (Euler's Constant)-এর কাছাকাছি:

$$c(n) = 2H(n - 1) - 2(n - 1)/n$$

Score-এর ব্যাখ্যা

- $s(x, n) \rightarrow 1$ -এর কাছাকাছি হলে সেই পয়েন্টটি অত্যন্ত সম্ভাব্য একটি Anomaly, কারণ এর Path Length খুবই ছোট (খুব কম Split-এই বিচ্ছিন্ন হয়ে গেছে)।
- $s(x, n) \rightarrow 0.5$ -এর কাছাকাছি হলে সেই পয়েন্টটি সম্পূর্ণ ডেটাসেটের একটি স্বাভাবিক বা Normal অংশ হিসেবে বিবেচিত হয়, কারণ এর Path Length গড় Path Length-এর কাছাকাছি।
- $s(x, n) 0.5$ -এর চেয়েও ছোট হলে, সেই পয়েন্ট নিশ্চিতভাবে Normal ডেটার একদম কেন্দ্রে বা ঘন এলাকায় অবস্থিত, যেখানে তাকে আলাদা করা সবচেয়ে কঠিন।

গাণিতিক অন্তর্দৃষ্টি: $c(n)$ মূলত একটি Average-Case Normalization যা একটি সাধারণ, ব্যর্থ (Unsuccessful search-এর মতো আচরণকারী) Binary Search Tree-তে n সংখ্যক পয়েন্টকে বিচ্ছিন্ন করতে প্রত্যাশিত গড় Path Length প্রতিনিধিত্ব করে। এই Normalization-এর কারণেই Anomaly Score বিভিন্ন আকারের ডেটাসেট জুড়ে তুলনায়োগ্য (Comparable) থাকে — $n = 100$ এবং $n = 100,000$ হলেও Score-এর ব্যাখ্যা একই থাকে (0.5 মানে Normal, 1-এর কাছাকাছি মানে Anomaly)।

২২.৬ গুরুত্বপূর্ণ হাইপারপ্যারামিটার – বিস্তারিত ব্যাখ্যা

n_estimators — Default: 100

কাজ কী: মোট কতগুলো Isolation Tree তৈরি হবে তা নির্ধারণ করে। কখন Default রাখব: ১০০টি গাছ সাধারণত একটি স্থিতিশীল ও নির্ভরযোগ্য গড় Path Length পেতে যথেষ্ট। কখন পরিবর্তন করব: যদি ফলাফল Run-থেকে-Run অস্থির মনে হয়, তবে $n_estimators$ বাড়িয়ে (যেমন ২০০-৩০০) স্থিতিশীলতা বাড়ানো যায়, যদিও এতে গণনার সময় সামান্য বাড়বে।

max_samples — Default: 'auto' (min(256, n))

কাজ কী: প্রতিটি গাছ তৈরির সময় সম্পূর্ণ ডেটাসেট থেকে Randomly কতগুলো Sample ব্যবহার করা হবে তা নির্ধারণ করে। কখন Default রাখব: গবেষণায় দেখা গেছে যে মাত্র ২৫৬টি Sample-ই সাধারণত একটি নির্ভরযোগ্য Isolation

Tree তৈরি করতে যথেষ্ট, তাই এই ছোট Default মান দ্রুতগতির জন্য বিশেষভাবে বেছে নেওয়া হয়েছে। কখন পরিবর্তন করব: ডেটাসেট অত্যন্ত ছোট হলে এই মান সম্পূর্ণ ডেটাসেটের সমান বা কাছাকাছি রাখা যেতে পারে।

contamination — Default: 'auto'

কাজ কী: ডেটাসেটে আনুমানিক কত শতাংশ Anomaly আছে তা নির্দিষ্ট করে, যা predict() মেথডে Normal (1) বা Anomaly (-1) নির্ধারণের Decision Threshold ঠিক করতে ব্যবহৃত হয়। কখন পরিবর্তন করব: যদি ব্যবসায়িক প্রেক্ষাপট থেকে আনুমানিক জানা থাকে যে ডেটার প্রায় ২% Fraud বা Anomaly, তবে contamination=0.02 সেট করলে Threshold সেই অনুপাতে সামঞ্জস্য হবে।

Parameter	Default	সংক্ষিপ্ত কাজ
max_features	1.0	প্রতিটি Split-এর সময় সর্বোচ্চ কতগুলো ফিচার বিবেচনা করা হবে।
bootstrap	False	Sample গুলো Replacement সহ (Bootstrap) নাকি ছাড়া নেওয়া হবে তা নির্ধারণ করে।
random_state	None	ফলাফল পুনরুৎপাদনযোগ্য (Reproducible) রাখার জন্য একটি ফিক্সড সংখ্যা দেওয়া উচিত (যেমন 42)।

Best Practice: Production-এ contamination='auto' দিয়ে শুরু করা এবং score_samples() থেকে পাওয়া Continuous Score-এর Distribution পর্যবেক্ষণ করে ব্যবসায়িক প্রয়োজন অনুযায়ী নিজস্ব Threshold ঠিক করাই সবচেয়ে নির্ভরযোগ্য পদ্ধতি — কারণ বাস্তব Contamination হার প্রায়ই সময়ের সাথে পরিবর্তিত হয়।

২২.৭ গুরুত্বপূর্ণ অ্যাট্রিবিউট ও মেথড

- predict(X) : প্রতিটি পয়েন্টকে Normal (1) নাকি Anomaly (-1) হিসেবে চিহ্নিত করে।
- decision_function(X) : প্রতিটি পয়েন্টের জন্য একটি Continuous Score দেয় — যত ঋণাত্মক (Negative), তত বেশি Anomalous, যা Ranking-এর জন্য ব্যবহার করা যায়।
- score_samples(X) : প্রতিটি পয়েন্টের র-অ্যানোমালি স্কোর (Raw Anomaly Score) প্রদান করে।
- estimators_ : ট্রেনিং হওয়া সবগুলো পৃথক Isolation Tree-এর অ্যারে।

২২.৮ কোথায় ব্যবহার হয়?

- ফাইন্যান্স ও ব্যাংকিং: ক্রেডিট কার্ড লেনদেনে জালিয়াতি (Fraud) শনাক্তকরণ — অস্বাভাবিক পরিমাণ বা অস্বাভাবিক লোকেশনের লেনদেন দ্রুত চিহ্নিত করা।
- নেটওয়ার্ক সিকিউরিটি: Intrusion Detection — অস্বাভাবিক নেটওয়ার্ক ট্রাফিক প্যাটার্ন বা Cyber Attack শনাক্ত করা।

- শিল্প ও উৎপাদন (Manufacturing): সেন্সর ডেটায় যন্ত্রের অস্বাভাবিক আচরণ শনাক্ত করে যন্ত্র বিকল হওয়ার আগেই Predictive Maintenance করা।
- স্বাস্থ্যসেবা: রোগীর হার্টল সাইনে (ECG, Blood Pressure) অস্বাভাবিক প্যাটার্ন প্রাথমিকভাবে ফ্ল্যাগ করে চিকিৎসকের নজরে আনা।
- ই-কমার্স: রিভিউ স্প্যাম বা বট-চালিত ভুয়া অর্ডার প্যাটার্ন শনাক্তকরণ, যেখানে Labeled ডেটা পাওয়া কঠিন।
- IoT ও Smart Grid: বিদ্যুৎ ব্যবহারের সেন্সর ডেটায় অস্বাভাবিক খরচের প্যাটার্ন শনাক্ত করে বিদ্যুৎ চুরি বা যন্ত্র ত্রুটি নির্ণয়।

প্রশ্ন: Isolation Forest-এ কেন Gini বা Entropy-এর মতো Impurity পরিমাপের প্রয়োজন হয় না?

উত্তর: সাধারণ Decision Tree-এর লক্ষ্য থাকে ডেটাকে Class অনুযায়ী নিখুঁতভাবে আলাদা করা, তাই সেখানে Gini বা Entropy দিয়ে 'সেরা' Split খুঁজে বের করতে হয়। কিন্তু Isolation Forest-এর লক্ষ্য সম্পূর্ণ ভিন্ন — এটি শুধু জানতে চায় একটি পয়েন্টকে বিচ্ছিন্ন করতে কতগুলো Random Split প্রয়োজন হয়। যেহেতু Anomaly পয়েন্টগুলো এমনভাবেই সংখ্যায় কম ও বাকিদের থেকে ভিন্ন, তাই সম্পূর্ণ Random Split ব্যবহার করলেও তারা স্বাভাবিকভাবেই দ্রুত বিচ্ছিন্ন হয়ে যায় — এই কারণে জটিল Impurity গণনার প্রয়োজন পড়ে না, যা অ্যালগরিদমকে অনেক দ্রুতগতির করে তোলে।

প্রশ্ন: Isolation Forest কি High-Dimensional ডেটাসেটে (যেমন শত শত ফিচার) ভালো কাজ করে?

উত্তর: সাধারণভাবে হ্যাঁ, কারণ এটি Distance বা Density গণনার ওপর নির্ভর করে না, তাই Curse of Dimensionality-এর প্রভাব তুলনামূলকভাবে কম পড়ে। তবে ফিচার সংখ্যা অত্যধিক বেশি হলে (যেমন কয়েক হাজার) এবং তার মধ্যে প্রাসঙ্গিক (Relevant) ফিচার কম হলে, Random Feature নির্বাচনের কারণে গুরুত্বপূর্ণ ফিচারগুলো Split-এ কম বাছাই হতে পারে; এই ক্ষেত্রে প্রথমে Feature Selection বা PCA প্রয়োগ করে অপ্রাসঙ্গিক ফিচার কমিয়ে আনা একটি ভালো Best Practice।

২২.৯ সাধারণ ভুল ধারণা ও ডিবাগিং টিপস

- ভুল ধারণা: 'Isolation Forest সবসময় ফিচার স্কেলিং-মুক্ত।' — বাস্তবে এটি সত্য যে Isolation Forest Distance-নির্ভর নয় বলে সাধারণ StandardScaler প্রয়োজন হয় না, তবে যদি কিছু ফিচারের ভ্যালু-রেঞ্জ অস্বাভাবিকভাবে বড় হয় এবং $\max_features < 1.0$ সেট করা থাকে, তাহলে ফিচার নির্বাচনের র্যান্ডমনেসে সামান্য প্রভাব পড়তে পারে।
- ভুল ধারণা: 'Anomaly Score সবসময় 0 থেকে 1-এর মধ্যে থাকা মানেই সহজে ব্যাখ্যাযোগ্য।' — বাস্তবে `decision_function()`-এর মান এবং `score_samples()`-এর মান ভিন্ন Convention অনুসরণ করে (একটিতে Negative মানে Anomaly, অন্যটিতে সরাসরি Raw Score); ডকুমেন্টেশন ভালোভাবে যাচাই না করে দুটি মেথডের আউটপুট গুলিয়ে ফেলা একটি সাধারণ ভুল।
- ডিবাগিং টিপ: যদি প্রায় সব পয়েন্টের Anomaly Score কাছাকাছি মনে হয় (Model কিছুই আলাদা করতে পারছে না), তাহলে `max_samples` বাড়িয়ে বা `n_estimators` বাড়িয়ে দেখুন, এবং ফিচারগুলোর মধ্যে Redundant বা Highly-Correlated ফিচার আছে কিনা যাচাই করুন — এরকম ফিচার Random Split-এর কার্যকারিতা কমিয়ে দিতে পারে।

২২.১০ সম্পূর্ণ ম্যানুয়াল অ্যালগরিদম এক্সিকিউশন ও Python ইমপ্লিমেন্টেশন

এই সেকশনে আমরা "আমরাই কম্পিউটার" এই দৃষ্টিভঙ্গি নিয়ে সম্পূর্ণ Isolation Forest অ্যালগরিদমটি হাতে-কলমে, প্রতিটি Split, প্রতিটি তুলনা এবং প্রতিটি গাণিতিক ধাপ সম্পূর্ণভাবে দেখিয়ে, একেবারে শুরু থেকে শেষ পর্যন্ত কার্যকর করব। এরপর সম্পূর্ণ From-Scratch Python কোড, তারপর সম্পূর্ণ Scikit-learn ইমপ্লিমেন্টেশন, এবং সবশেষে একটি নতুন পয়েন্টের ওপর প্রেডিকশন ও তার ব্যাখ্যা দেওয়া হবে।

২২.১০.১ একটি সম্পূর্ণ Isolation Tree-এর সম্পূর্ণ ম্যানুয়াল নির্মাণ

আমরা একই ডেটাসেট ব্যবহার করছি: {10, 12, 11, 13, 9, 500} (n = 6)। এবার আমরা প্রতিটি একক পয়েন্টের জন্য সম্পূর্ণ Path Length বের করব, কোনো ধাপ বাদ না দিয়ে। ধরে নিই random_state এমনভাবে নির্ধারিত হয়েছে যে প্রতিটি Node-এ নিচের নির্দিষ্ট Random Split Value বেছে নেওয়া হয়েছে (বাস্তবে এই মানগুলো uniform(min, max) থেকে র‍্যান্ডমভাবে আসে, কিন্তু ম্যানুয়াল পুনরুৎপাদনযোগ্যতার জন্য আমরা নির্দিষ্ট মান ধরে নিচ্ছি)।

Depth 0 (Root Node)

Node-এর ডেটা: {10, 12, 11, 13, 9, 500} — মোট ৬টি পয়েন্ট। এই ফিচারের Minimum = 9, Maximum = 500। একটি Random Split Value বেছে নেওয়া হলো: split_0 = 254.3 (যা 9 এবং 500-এর মধ্যে একটি র‍্যান্ডম সংখ্যা)।

এবার প্রতিটি পয়েন্টকে এই Threshold-এর সাথে তুলনা করা হচ্ছে:

- $10 \leq 254.3 \rightarrow$ বামে যাবে
- $12 \leq 254.3 \rightarrow$ বামে যাবে
- $11 \leq 254.3 \rightarrow$ বামে যাবে
- $13 \leq 254.3 \rightarrow$ বামে যাবে
- $9 \leq 254.3 \rightarrow$ বামে যাবে
- $500 > 254.3 \rightarrow$ ডানে যাবে

ফলাফল: বাম Node = {10, 12, 11, 13, 9} (৫টি পয়েন্ট), ডান Node = {500} (১টি পয়েন্ট)। যেহেতু ডান Node-এ মাত্র ১টি পয়েন্ট আছে, এই Node একটি Leaf হিসেবে গণ্য হয় এবং আর ভাগ হবে না। অতএব, 500 পয়েন্টটি Depth 1-এ সম্পূর্ণভাবে বিচ্ছিন্ন (Isolated) হয়ে গেছে। সুতরাং $h(500) = 1$ । এটি একটি চূড়ান্ত ফলাফল, কোনো অনুমান নয় — Root থেকে এই Leaf পর্যন্ত পৌঁছাতে ঠিক ১টি Split লেগেছে।

Depth 1 (বাম Node-এর বিভাজন)

Node-এর ডেটা: {10, 12, 11, 13, 9} — মোট ৫টি পয়েন্ট। এই ফিচারের Minimum = 9, Maximum = 13। একটি Random Split Value বেছে নেওয়া হলো: split_1 = 11.4।

প্রতিটি পয়েন্টকে তুলনা করা হচ্ছে:

- $10 \leq 11.4 \rightarrow$ বামে যাবে

- $12 \leq 11.4$? না, $12 > 11.4 \rightarrow$ ডানে যাবে
- $11 \leq 11.4 \rightarrow$ বামে যাবে
- $13 > 11.4 \rightarrow$ ডানে যাবে
- $9 \leq 11.4 \rightarrow$ বামে যাবে

ফলাফল: বাম Node = {10, 11, 9} (৩টি পয়েন্ট), ডান Node = {12, 13} (২টি পয়েন্ট)। উভয় Node-এই ১টির বেশি পয়েন্ট আছে, তাই উভয়ই আরও ভাগ হবে। এখনো পর্যন্ত কোনো নতুন পয়েন্ট Isolate হয়নি এই ধাপে।

Depth 2 (বাম-বাম Node: {10, 11, 9}-এর বিভাজন)

Node-এর ডেটা: {10, 11, 9} — মোট ৩টি পয়েন্ট। Minimum = 9, Maximum = 11। Random Split Value: split_2a = 9.8।

- $10 \leq 9.8$? না, $10 > 9.8 \rightarrow$ ডানে যাবে
- $11 > 9.8 \rightarrow$ ডানে যাবে
- $9 \leq 9.8 \rightarrow$ বামে যাবে

ফলাফল: বাম Node = {9} (১টি পয়েন্ট, তাৎক্ষণিক Leaf), ডান Node = {10, 11} (২টি পয়েন্ট, আরও ভাগ হবে)। যেহেতু Root (Depth 0) $\rightarrow$ এই Node (Depth 1) $\rightarrow$ এই Split (Depth 2)-এর মাধ্যমে ৩ বিচ্ছিন্ন হয়েছে, তাই $h(9) = 3$ (মোট ৩টি Split অতিক্রম করে এই পয়েন্ট পর্যন্ত পৌঁছাতে এবং তাকে বিচ্ছিন্ন করতে হয়েছে: Split_0, Split_1, Split_2a)।

Depth 3 (বাম-বাম-ডান Node: {10, 11}-এর বিভাজন)

Node-এর ডেটা: {10, 11} — মোট ২টি পয়েন্ট। Minimum = 10, Maximum = 11। Random Split Value: split_3 = 10.6।

- $10 \leq 10.6 \rightarrow$ বামে যাবে
- $11 > 10.6 \rightarrow$ ডানে যাবে

ফলাফল: বাম Node = {10} (Leaf), ডান Node = {11} (Leaf)। উভয় পয়েন্টই এই একক Split-এর মাধ্যমে একে অপর থেকে সম্পূর্ণ বিচ্ছিন্ন হয়ে গেছে। এই Split পর্যন্ত পৌঁছাতে মোট ৪টি Split অতিক্রম করতে হয়েছে (Split_0, Split_1, Split_2a, Split_3), তাই $h(10) = 4$ এবং $h(11) = 4$ ।

Depth 2 (বাম-ডান Node: {12, 13}-এর বিভাজন)

Node-এর ডেটা: {12, 13} — মোট ২টি পয়েন্ট। Minimum = 12, Maximum = 13। Random Split Value: split_2b = 12.5।

- $12 \leq 12.5 \rightarrow$ বামে যাবে
- $13 > 12.5 \rightarrow$ ডানে যাবে

ফলাফল: বাম Node = {12} (Leaf), ডান Node = {13} (Leaf)। এই Split পর্যন্ত পৌঁছাতে মোট ৩টি Split অতিক্রম করতে হয়েছে (Split_0, Split_1, Split_2b), তাই $h(12) = 3$ এবং $h(13) = 3$ ।

সম্পূর্ণ গাছের গঠন যাচাই: এই মুহূর্তে সবগুলো পয়েন্ট (10, 12, 11, 13, 9, 500) সম্পূর্ণভাবে পৃথক Leaf-এ বিচ্ছিন্ন হয়ে গেছে, এবং কোনো পয়েন্ট বাদ পড়েনি – এটি নিশ্চিত করে যে গাছটি সম্পূর্ণভাবে তৈরি হয়েছে। প্রাপ্ত Path Length-গুলো হলো: $h(500)=1$, $h(9)=3$, $h(10)=4$, $h(11)=4$, $h(12)=3$, $h(13)=3$ ।

পয়েন্ট (x)	Path Length $h(x)$ — একটি গাছে	যে Split-গুলো অতিক্রম করেছে
500	1	Split_0
9	3	Split_0 → Split_1 → Split_2a
10	4	Split_0 → Split_1 → Split_2a → Split_3
11	4	Split_0 → Split_1 → Split_2a → Split_3
12	3	Split_0 → Split_1 → Split_2b
13	3	Split_0 → Split_1 → Split_2b

২২.১০.২ প্রতিটি পয়েন্টের জন্য সম্পূর্ণ Anomaly Score গণনা (একক গাছ থেকে)

আমরা ইতিমধ্যে $c(6)$ গণনা করেছি। পুনরায় ধাপে ধাপে দেখানো হলো:

$$\begin{aligned}
 H(5) &= 1/1 + 1/2 + 1/3 + 1/4 + 1/5 \\
 &= 1.0000 + 0.5000 + 0.3333 + 0.2500 + 0.2000 \\
 &= 2.2833 \\
 c(6) &= 2 \times H(5) - (2 \times (6 - 1)) / 6 \\
 &= 2 \times 2.2833 - (2 \times 5) / 6 \\
 &= 4.5667 - 10/6 \\
 &= 4.5667 - 1.6667 \\
 &= 2.9000
 \end{aligned}$$

এখন $h = 1$ (পয়েন্ট 500)-এর জন্য সম্পূর্ণ ধাপে ধাপে গণনা:

$$\begin{aligned}
 s(500, 6) &= 2^{(-h/c(n))} = 2^{(-1 / 2.9000)} \\
 \text{exponent} &= -1 / 2.9000 = -0.3448 \\
 2^{(-0.3448)} &= 1 / 2^{(0.3448)} \\
 2^{(0.3448)} &= e^{(0.3448 \times \ln 2)} = e^{(0.3448 \times 0.6931)} = e^{(0.2390)} = 1.2699 \\
 s(500, 6) &= 1 / 1.2699 = 0.7875
 \end{aligned}$$

এখন $h = 3$ (পয়েন্ট 9, 12, এবং 13 — তিনটিরই h একই, তাই এদের জন্য গণনার ধাপ অভিন্ন এবং ফলাফলও অভিন্ন হবে; এটি কোনো ধাপ বাদ দেওয়া নয়, বরং একই ইনপুট থেকে গাণিতিকভাবে একই আউটপুট আসা আবশ্যিক):

$$s(x, 6) = 2^{(-3 / 2.9000)}$$

$$\text{exponent} = -3 / 2.9000 = -1.0345$$

$$2^{(1.0345)} = e^{(1.0345 \times 0.6931)} = e^{(0.7171)} = 2.0486$$

$$s(x, 6) = 1 / 2.0486 = 0.4881$$

অতএব $s(9, 6) = 0.4881$, $s(12, 6) = 0.4881$, $s(13, 6) = 0.4881$ ।

এখন $h = 4$ (পয়েন্ট 10 এবং 11):

$$s(x, 6) = 2^{(-4 / 2.9000)}$$

$$\text{exponent} = -4 / 2.9000 = -1.3793$$

$$2^{(1.3793)} = e^{(1.3793 \times 0.6931)} = e^{(0.9560)} = 2.6013$$

$$s(x, 6) = 1 / 2.6013 = 0.3844$$

অতএব $s(10, 6) = 0.3844$, $s(11, 6) = 0.3844$ ।

পয়েন্ট	$h(x)$	Anomaly Score $s(x, 6)$	সিদ্ধান্ত
500	1	0.7875	Anomaly (Score 1-এর কাছাকাছি)
9	3	0.4881	Normal
10	4	0.3844	Normal (গভীরভাবে Normal)
11	4	0.3844	Normal (গভীরভাবে Normal)
12	3	0.4881	Normal
13	3	0.4881	Normal

২২.১০.৩ Ensemble Averaging-এর সম্পূর্ণ ম্যানুয়াল উদাহরণ (একাধিক গাছ)

বাস্তবে $n_{\text{estimators}}=100$ বা তার বেশি গাছ তৈরি হয়, প্রতিটি গাছে ভিন্ন Random Split হওয়ায় একই পয়েন্টের Path Length গাছে গাছে সামান্য ভিন্ন হতে পারে। এখানে একটি ছোট, ৩-গাছের Mini-Forest ধরে পয়েন্ট 500-এর জন্য সম্পূর্ণ Ensemble Averaging দেখানো হলো, যেখানে প্রতিটি গাছে ভিন্ন Random Split-এর কারণে ভিন্ন Path Length পাওয়া গেছে (এটি বাস্তবসম্মত, কারণ Randomness-এর কারণে ফলাফল গাছে গাছে সামান্য পরিবর্তিত হয়):

গাছ	500-এর Path Length $h(500)$
Tree 1	1
Tree 2	2
Tree 3	1

$$\begin{aligned}
 E(h(500)) &= (h_Tree1 + h_Tree2 + h_Tree3) / 3 \\
 &= (1 + 2 + 1) / 3 \\
 &= 4 / 3 \\
 &= 1.3333
 \end{aligned}$$

$$\begin{aligned}
 s(500, 6) &= 2^{(-1.3333 / 2.9000)} \\
 \text{exponent} &= -1.3333 / 2.9000 = -0.4598 \\
 2^{(0.4598)} &= e^{(0.4598 \times 0.6931)} = e^{(0.3187)} = 1.3754 \\
 s(500, 6) &= 1 / 1.3754 = 0.7271
 \end{aligned}$$

পর্যবেক্ষণ: লক্ষ করুন, একক গাছে (Score=0.7875) এবং তিন-গাছের Ensemble-এ (Score=0.7271) ফলাফল সামান্য ভিন্ন, কিন্তু উভয় ক্ষেত্রেই Score 0.5-এর অনেক ওপরে থাকায় সিদ্ধান্ত অপরিবর্তিত থাকে — 500 একটি Anomaly। এটিই দেখায় কেন Ensemble Averaging প্রয়োজন: এটি একটি একটি একক গাছের Randomness-জনিত ওঠানামা কমিয়ে একটি অধিকতর স্থিতিশীল চূড়ান্ত সিদ্ধান্তে পৌঁছাতে সাহায্য করে।

২২.১০.৪ From-Scratch Python ইমপ্লিমেন্টেশন (সম্পূর্ণ, কোনো অংশ বাদ নেই)

নিচে Isolation Forest-এর একটি সম্পূর্ণ From-Scratch Implementation দেওয়া হলো। প্রতিটি লাইনে মন্তব্য (Comment) যোগ করা হয়েছে যাতে বোঝা যায় কোডের প্রতিটি অংশ ঠিক কী করছে এবং কেন করছে।

```
import numpy as np

class IsolationTreeNode:
    """একটি একক Node - হয় এটি একটি Leaf, অথবা এর বাম ও ডান সন্তান আছে।"""
    def __init__(self):
        self.is_leaf = True          # ডিফল্টভাবে প্রতিটি Node একটি Leaf হিসেবে শুরু হয়
        self.split_feature = None     # কোন ফিচারে Split করা হয়েছে
        self.split_value = None       # Split-এর Threshold মান
        self.left = None              # বাম সন্তান Node
        self.right = None             # ডান সন্তান Node
        self.size = 0                 # এই Node-এ কতগুলো পয়েন্ট আছে (Leaf হলে দরকার হয়)

def build_tree(X, current_depth, max_depth):
    """
    X: বর্তমান Node-এ থাকা ডেটা পয়েন্টগুলোর অ্যারে (n_samples, n_features)
    current_depth: এই Node-এর গভীরতা (Root = 0)
    max_depth: সর্বোচ্চ অনুমোদিত গভীরতা (log2(sample_size) থেকে গণনা করা হয়)
    রিটার্ন করে: একটি IsolationTreeNode
    """
    node = IsolationTreeNode()
    node.size = X.shape[0]

    # থামার শর্ত ১: যদি Node-এ ১টির কম বা সমান পয়েন্ট থাকে, এটি একটি Leaf
    # থামার শর্ত ২: যদি সর্বোচ্চ গভীরতায় পৌঁছে যাওয়া হয়
    if X.shape[0] <= 1 or current_depth >= max_depth:
        node.is_leaf = True
        return node
```

```

n_features = X.shape[1]
# ধাপ ১: একটি র‍্যান্ডম ফিচার বেছে নেওয়া (Gini/Entropy নয়, সম্পূর্ণ Random)
split_feature = np.random.randint(0, n_features)

feature_values = X[:, split_feature]
min_val = feature_values.min()
max_val = feature_values.max()

# যদি সব মান একই হয় (Min == Max), Split সম্ভব নয়, তাই এটি একটি Leaf
if min_val == max_val:
    node.is_leaf = True
    return node

# ধাপ ২: Min ও Max-এর মধ্যে একটি সম্পূর্ণ Random Split Value বেছে নেওয়া
split_value = np.random.uniform(min_val, max_val)

# ধাপ ৩: ডেটাকে দুই ভাগে ভাগ করা
left_mask = feature_values <= split_value
right_mask = ~left_mask

left_data = X[left_mask]
right_data = X[right_mask]

# যদি এক পাশে কোনো পয়েন্ট না থাকে (সব পয়েন্ট একই দিকে গেছে), এটি একটি Leaf
if left_data.shape[0] == 0 or right_data.shape[0] == 0:
    node.is_leaf = True
    return node

# সফলভাবে Split হয়েছে, তাই এই Node একটি Internal Node (Leaf নয়)
node.is_leaf = False
node.split_feature = split_feature
node.split_value = split_value

# ধাপ ৪: পুনরাবৃত্তিমূলকভাবে (Recursively) বাম ও ডান Sub-tree তৈরি করা
node.left = build_tree(left_data, current_depth + 1, max_depth)
node.right = build_tree(right_data, current_depth + 1, max_depth)

return node

def path_length(x, node, current_depth):
    """
    একটি একক পয়েন্ট x-কে গাছের Root থেকে নিচে নামিয়ে
    তার Path Length (কতগুলো Split অতিক্রম করে Isolate হলো) গণনা করে।
    """
    # যদি এখন Leaf-এ পৌঁছে যাই, তবে বর্তমান গভীরতা ফেরত দাও
    # (node.size > 1 হলে একটি ছোট Correction যোগ করা হয়, যা c(size) দিয়ে
    # অসম্পূর্ণভাবে ভাগ হওয়া Node-এর অবশিষ্ট গড় Path Length অনুমান করে)
    if node.is_leaf:
        if node.size > 1:
            return current_depth + average_path_length(node.size)
        return current_depth

    # নির্দিষ্ট Split Feature-এর মান বের করা
    value = x[node.split_feature]

    # তুলনা করে বামে বা ডানে যাওয়া (ঠিক যেমন Training-এর সময় হয়েছিল)
    if value <= node.split_value:

```

```

        return path_length(x, node.left, current_depth + 1)
    else:
        return path_length(x, node.right, current_depth + 1)

def harmonic_number(i):
    """H(i) = 1 + 1/2 + 1/3 + ... + 1/i - সরাসরি সংজ্ঞা অনুযায়ী গণনা।"""
    total = 0.0
    for k in range(1, int(i) + 1):
        total += 1.0 / k
    return total

def average_path_length(n):
    """c(n) = 2*H(n-1) - 2*(n-1)/n - Normalization ফ্যাক্টর।"""
    if n <= 1:
        return 0.0
    return 2.0 * harmonic_number(n - 1) - (2.0 * (n - 1) / n)

class IsolationForestFromScratch:
    """সম্পূর্ণ From-Scratch Isolation Forest - sklearn-এর মতো fit/predict API সহ।"""

    def __init__(self, n_estimators=100, max_samples=256, contamination=0.1,
                 random_state=None):
        self.n_estimators = n_estimators          # মোট কতগুলো গাছ তৈরি হবে
        self.max_samples = max_samples            # প্রতিটি গাছে কতগুলো Sample ব্যবহার হবে
        self.contamination = contamination        # আনুমানিক Anomaly অনুপাত (Threshold-এর জন্য)
        self.trees = []                          # তৈরি হওয়া সবগুলো গাছ এখানে সংরক্ষিত হবে
        if random_state is not None:
            np.random.seed(random_state)          # ফলাফল Reproducible রাখার জন্য

    def fit(self, X):
        n_samples = X.shape[0]
        # প্রতিটি গাছের জন্য সর্বোচ্চ কতগুলো Sample ব্যবহার হবে তা নির্ধারণ
        sample_size = min(self.max_samples, n_samples)
        # সর্বোচ্চ গভীরতা: ceil(log2(sample_size)) - গাছকে অতিরিক্ত গভীর হতে বাধা দেয়
        self.max_depth = int(np.ceil(np.log2(sample_size))) if sample_size > 1 else 1
        self.trees = []
        for i in range(self.n_estimators):
            # প্রতিটি গাছের জন্য আলাদা Random Sub-sample বেছে নেওয়া (Bootstrap ছাড়া)
            indices = np.random.choice(n_samples, size=sample_size, replace=False)
            sample = X[indices]
            tree = build_tree(sample, current_depth=0, max_depth=self.max_depth)
            self.trees.append(tree)
        # Threshold গণনার জন্য Training ডেটার ওপর Score-গুলো সংরক্ষণ করা
        self._training_scores = self.score_samples(X)
        # contamination অনুপাত অনুযায়ী Threshold ঠিক করা
        self.threshold_ = np.percentile(self._training_scores, 100 * (1 - self.contamination))
        return self

    def score_samples(self, X):
        """প্রতিটি পয়েন্টের জন্য Anomaly Score s(x, n) গণনা করে।"""
        c_n = average_path_length(len(self._reference_n) if hasattr(self, '_reference_n') else
                                   self.max_samples)
        scores = np.zeros(X.shape[0])
        for idx in range(X.shape[0]):
            x = X[idx]
            # সবগুলো গাছ জুড়ে গড় Path Length বের করা
            path_lengths = [path_length(x, tree, 0) for tree in self.trees]
            avg_path_length = np.mean(path_lengths)

```

```
# সূত্র প্রয়োগ:  $s(x,n) = 2^{(-E(h(x))/c(n))}$ 
scores[idx] = 2.0 ** (-avg_path_length / c_n)
return scores

def predict(self, X):
    """1 = Normal, -1 = Anomaly - Threshold-এর সাথে তুলনা করে।"""
    scores = self.score_samples(X)
    return np.where(scores >= self.threshold_, -1, 1)
```

লাইন-বাই-লাইন গুরুত্বপূর্ণ ব্যাখ্যা: build_tree() ফাংশনটি প্রতিটি Node-এ ঠিক আমরা যে ম্যানুয়াল উদাহরণে করেছি সেই একই যুক্তি অনুসরণ করে — একটি Random ফিচার, একটি Random Threshold, এবং তারপর তুলনা করে বাম/ডান ভাগ করা। path_length() ফাংশনটি ঠিক আমাদের ম্যানুয়াল উদাহরণে দেখানো Root থেকে Leaf পর্যন্ত হাঁটার প্রক্রিয়াটিই কোডে রূপান্তরিত করে। average_path_length() ফাংশনটি ঠিক $c(n)$ সূত্রটিই বাস্তবায়ন করে, ধাপে ধাপে $H(n-1)$ গণনা করে।

সাধারণ শিক্ষানবিশ ভুল (Beginner Mistakes)

- ভুল: sample_size-এর বদলে সম্পূর্ণ n_samples ব্যবহার করে $c(n)$ গণনা করা। সঠিক: $c(n)$ -এর n হওয়া উচিত প্রতিটি গাছে ব্যবহৃত sample_size, সম্পূর্ণ ডেটাসেটের আকার নয় — কারণ প্রতিটি গাছ শুধুমাত্র sample_size সংখ্যক পয়েন্টের ওপর তৈরি হয়েছে।
- ভুল: max_depth সেট না করে গাছকে সম্পূর্ণভাবে Leaf পর্যন্ত বাড়তে দেওয়া (বিশেষত বড় ডেটাসেটে), যা অপ্রয়োজনীয়ভাবে গণনার সময় বাড়িয়ে দেয় — Isolation Forest-এর একটি মূল সুবিধাই হলো Early Stopping-এর মাধ্যমে গতি বৃদ্ধি করা।
- ভুল: predict()-এ Threshold হিসেবে সরাসরি 0.5 ব্যবহার করা, contamination-ভিত্তিক Percentile Threshold ব্যবহার না করে — এতে ভুল সংখ্যক পয়েন্ট Anomaly হিসেবে চিহ্নিত হতে পারে।

২২.১০.৫ সম্পূর্ণ Scikit-learn ইমপ্লিমেন্টেশন

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.datasets import make_blobs

# ধাপ ১: ডেটাসেট তৈরি করা - ২০০টি স্বাভাবিক পয়েন্ট + ১০টি ইচ্ছাকৃত Anomaly
X_normal, _ = make_blobs(n_samples=200, centers=[[0, 0]], cluster_std=1.0, random_state=42)
rng = np.random.RandomState(42)
X_anomalies = rng.uniform(low=-8, high=8, size=(10, 2)) # ব্যাপকভাবে ছড়িয়ে থাকা Anomaly পয়েন্ট
X = np.vstack([X_normal, X_anomalies]) # সম্পূর্ণ ডেটাসেট একত্রিত করা

# ধাপ ২: মডেল তৈরি ও ট্রেন করা
model = IsolationForest(
    n_estimators=100, # ১০০টি গাছ ব্যবহার করা হবে
    max_samples='auto', # প্রতিটি গাছে min(256, n) স্যাম্পল
    contamination=0.05, # আনুমানিক প্রায় ৫% ডেটা Anomaly ধরা হচ্ছে
    random_state=42 # Reproducibility-এর জন্য
)
model.fit(X)

# ধাপ ৩: প্রতিটি পয়েন্টের জন্য প্রেডিকশন (1 = Normal, -1 = Anomaly)
```

```

predictions = model.predict(X)
print('মোট Anomaly সংখ্যা চিহ্নিত হয়েছে:', np.sum(predictions == -1))

# ধাপ ৪: Continuous Anomaly Score (Ranking-এর জন্য উপযোগী)
scores = model.decision_function(X) # যত ঋণাত্মক, তত বেশি Anomalous
print('প্রথম ৫টি পয়েন্টের Score:', scores[:5])

# ধাপ ৫: মূল্যায়ন – আমরা জানি প্রকৃত Anomaly কোনগুলো (শেষ ১০টি পয়েন্ট),
# তাই একটি সহজ মূল্যায়ন করা যাক
true_labels = np.array([1]*200 + [-1]*10) # 1=Normal, -1=Anomaly (প্রকৃত লেবেল)
accuracy = np.mean(predictions == true_labels)
print('Accuracy (এই টয় উদাহরণে):', accuracy)

```

প্রতিটি প্যারামিটার এবং আউটপুট-এর ব্যাখ্যা: $n_estimators=100$ মানে ১০০টি স্বাধীন Isolation Tree তৈরি হবে এবং তাদের গড় Path Length থেকে চূড়ান্ত Score নির্ধারিত হবে। $contamination=0.05$ predict() মেথডের Decision Threshold ঠিক করে, যাতে আনুমানিক ৫% পয়েন্টকে -1 (Anomaly) হিসেবে চিহ্নিত করা হয়। decision_function()-এর আউটপুট একটি Continuous মান, যেখানে ঋণাত্মক মান Anomaly এবং ধনাত্মক মান Normal নির্দেশ করে — এই মানগুলো ব্যবহার করে পয়েন্টগুলোকে Anomaly-এর মাত্রা অনুযায়ী Rank করা সম্ভব, যা শুধুমাত্র Binary predict() আউটপুটের চেয়ে অনেক বেশি তথ্যবহুল।

২২.১০.৬ সম্পূর্ণ নতুন একটি Sample-এর ওপর প্রেডিকশন ও ব্যাখ্যা

```

# একটি সম্পূর্ণ নতুন লেনদেন পয়েন্ট, যা Training-এ ছিল না
new_transaction = np.array([[15.0, 15.0]]) # মূল Cluster (0,0)-এর অনেক দূরে

new_prediction = model.predict(new_transaction)
new_score = model.decision_function(new_transaction)

print('প্রেডিকশন:', new_prediction) # আউটপুট: [-1]
print('Decision Score:', new_score) # আউটপুট: একটি স্পষ্ট ঋণাত্মক সংখ্যা

```

ব্যাখ্যা: $new_transaction = (15.0, 15.0)$ মূল স্বাভাবিক Cluster, যা (0,0)-এর আশেপাশে কেন্দ্রীভূত, তার থেকে অনেক দূরে অবস্থিত। ফলে Isolation Forest-এর গাছগুলোতে এই পয়েন্টটি মাত্র কয়েকটি Random Split-এই বাকি সব ডেটা থেকে বিচ্ছিন্ন হয়ে যাবে, অর্থাৎ এর Path Length ছোট হবে। predict() আউটপুট -1 নির্দেশ করে যে মডেল একে Anomaly হিসেবে চিহ্নিত করেছে, এবং decision_function()-এর ঋণাত্মক মান এই সিদ্ধান্তের আত্মবিশ্বাসের মাত্রা প্রকাশ করে — মান যত বেশি ঋণাত্মক, মডেল তত বেশি নিশ্চিত যে এটি একটি Anomaly।

২২.১১ Quick Revision

- Type: Unsupervised, Tree-ভিত্তিক Anomaly Detection Algorithm।
- মূল ধারণা: Anomaly পয়েন্ট সংখ্যায় কম ও ভিন্ন হওয়ায় Random Split-এর মাধ্যমে দ্রুত (কম Path Length-এ) বিচ্ছিন্ন হয়ে যায়।
- Score: গড় Path Length থেকে গণনা করা Anomaly Score 1-এর কাছাকাছি হলে Outlier, 0.5-এর কাছাকাছি হলে Normal।

- সুবিধা: High-Dimensional ডেটায় Scalable, Distance বা Density গণনার প্রয়োজন নেই বলে অত্যন্ত দ্রুতগতির।
- মূল হাইপারপ্যারামিটার: `n_estimators`, `max_samples`, `contamination`।
- সীমাবদ্ধতা: Local বা খুব সূক্ষ্ম (Subtle) Anomaly, যা বাকি ডেটার সাথে আংশিকভাবে মিশে থাকে, শনাক্ত করতে কম কার্যকর হতে পারে।

Chapter 23 — One-Class SVM

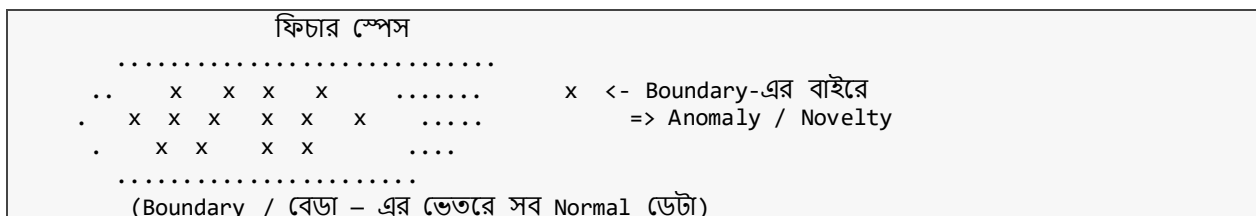
২৩.১ পরিচিতি ও দর্শন (Introduction and Philosophy)

One-Class SVM (Support Vector Machine) হলো একটি Unsupervised Anomaly বা Novelty Detection Algorithm, যা ঐতিহ্যবাহী Support Vector Machine-এর মূল ধারণার ওপর ভিত্তি করে তৈরি। এটি মূলত Bernhard Schölkopf এবং তাঁর সহকর্মীদের ১৯৯৯-২০০১ সালের কাজের মাধ্যমে প্রস্তাবিত হয়েছিল, যেখানে সাধারণ SVM-এর Margin-Maximization দর্শনকে একটি মাত্র Class-এর সমস্যায় প্রয়োগ করা হয়। সাধারণ SVM যেখানে দুই বা ততোধিক Class-কে (যেমন 'Spam' বনাম 'Not Spam') আলাদা করার জন্য একটি Decision Boundary শেখে, সেখানে One-Class SVM শুধুমাত্র একটি Class ('Normal' ডেটা) নিয়ে কাজ করে এবং সেই Normal ডেটাকে ঘিরে একটি নিখুঁত ও যতটা সম্ভব টাইট সীমানা (Boundary) তৈরি করার চেষ্টা করে।

এই পদ্ধতির শিল্পগত গুরুত্ব বিশেষভাবে সেসব ক্ষেত্রে দেখা যায় যেখানে Anomaly বা Novel ঘটনার কোনো পূর্ব উদাহরণ পাওয়া কঠিন — যেমন কোনো নতুন ধরনের Cyber Attack বা একেবারে নতুন ধরনের যান্ত্রিক ত্রুটি যা আগে কখনো দেখা যায়নি। এই ক্ষেত্রে শুধুমাত্র 'স্বাভাবিক' আচরণের ডেটা দিয়ে ট্রেনিং করে ভবিষ্যতে যেকোনো ধরনের বিচ্যুতি (Deviation) শনাক্ত করা সম্ভব হয়, তাই একে প্রায়ই Novelty Detection নামেও ডাকা হয়।

২৩.২ উদাহরণ (Intuition)

কল্পনা করুন, আপনি শুধুমাত্র সুস্থ মানুষদের হার্টবিট সাইন ডেটা (Heart Rate, Blood Pressure) দিয়ে একটি মডেল ট্রেনিং করছেন — অসুস্থ রোগীর কোনো ডেটা আপনার কাছে নেই, অথবা থাকলেও এতটাই কম যে তা দিয়ে একটি ভালো Classifier ট্রেনিং করা সম্ভব নয়। One-Class SVM এই সুস্থ মানুষদের ডেটার চারপাশে একটি টাইট, নিখুঁত সীমানা (Boundary) আঁকে, ঠিক যেন একটি অদৃশ্য 'বেড়া' দিয়ে সব স্বাভাবিক ডেটা পয়েন্টকে ঘিরে ফেলা হচ্ছে। ভবিষ্যতে নতুন কোনো রোগীর ডেটা যদি এই বেড়ার ভেতরে পড়ে, তবে তাকে 'স্বাভাবিক' বা Normal ধরা হয়; আর বেড়ার বাইরে পড়লে তাকে 'অস্বাভাবিক' বা Anomaly/Novelty হিসেবে চিহ্নিত করা হয়।



২৩.৩ মূল কার্যপ্রণালী (How it Works) — ধাপে ধাপে

১. Kernel Trick ব্যবহার

সাধারণ SVM-এর মতোই One-Class SVM ডেটাকে একটি উচ্চ-মাত্রিক (Higher-Dimensional) Feature Space-এ ম্যাপ করে, সাধারণত RBF (Radial Basis Function) বা Gaussian Kernel ব্যবহার করে। এর মাধ্যমে মূল স্পেসে জটিল ও Non-linear আকৃতির ডেটার জন্যও একটি কার্যকর সীমানা আঁকা সম্ভব হয়, যা সরাসরি মূল স্পেসে সম্ভব হতো না।

২. Origin থেকে সর্বোচ্চ দূরত্বে Hyperplane

এই উচ্চ-মাত্রিক স্পেসে অ্যালগরিদম এমন একটি Hyperplane (একটি সমতল বা সীমানা) খোঁজে, যা অধিকাংশ Training ডেটা পয়েন্টকে Origin (কেন্দ্রবিন্দু বা মূলবিন্দু) থেকে সর্বোচ্চ সম্ভব দূরত্বে আলাদা করে রাখে। এর অর্থ হলো, ডেটাকে Origin থেকে যতটা সম্ভব দূরে ঠেলে দিয়ে তাদের চারপাশে একটি যতটা সম্ভব টাইট সীমানা তৈরি করা হয়, যাতে সীমানার বাইরের এলাকা যতটা সম্ভব বড় থাকে — এই বড় বাইরের এলাকাটাই ভবিষ্যতে Anomaly শনাক্ত করার জন্য ব্যবহৃত হবে।

৩. Soft Margin (nu Parameter)

বাস্তব জীবনের ডেটায় প্রায়ই সব Training পয়েন্টকে নিখুঁতভাবে সীমানার ভেতরে রাখা সম্ভব হয় না (কারণ Training ডেটাতেও সামান্য Noise থাকতে পারে)। তাই nu প্যারামিটার দিয়ে নির্দিষ্ট করা হয় যে Training ডেটার সর্বোচ্চ কত শতাংশ পয়েন্টকে সীমানার বাইরে (Outlier হিসেবে) থাকতে দেওয়া 'অনুমোদিত' বা গ্রহণযোগ্য — এই নমনীয়তাকে বলা হয় Soft Margin।

৪. Decision Function

একবার Boundary শেখা হয়ে গেলে, নতুন যেকোনো পয়েন্টের জন্য অ্যালগরিদম গণনা করে সে এই শেখা সীমানার ভেতরে (Positive Side, অর্থাৎ Normal) নাকি বাইরে (Negative Side, অর্থাৎ Anomaly) অবস্থান করছে।

২৩.৪ গাণিতিক ভিত্তি – বিস্তারিত

One-Class SVM (Schölkopf-এর প্রস্তাবিত ν -SVM Formulation অনুযায়ী) নিচের Optimization সমস্যাটি সমাধান করে। এখানে w হলো Hyperplane-এর দিক নির্দেশক ভেক্টর, ρ (Rho) হলো Origin থেকে Hyperplane-এর দূরত্ব, ξ_i (X_i) হলো Slack Variable যা কিছু পয়েন্টকে সীমানার ভেতরে না থেকে একটু 'বাইরে' থাকার অনুমতি দেয়, এবং ν (Nu) $\in (0,1]$ হলো Outlier Fraction নিয়ন্ত্রণকারী প্যারামিটার:

$$\min (1/2)\|w\|^2 + (1/(\nu n)) \sum_i \xi_i - \rho$$

Subject to:

$$w \cdot \phi(x_i) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

এখানে $\phi(x)$ হলো Kernel Function দ্বারা মূল ডেটাকে উচ্চ-মাত্রিক স্পেসে Mapping করার একটি ফাংশন। এই Optimization সমস্যার লক্ষ্য হলো $\|w\|$ (Hyperplane-এর জটিলতা) সর্বনিম্ন রাখা এবং একই সাথে ρ (Origin থেকে দূরত্ব) সর্বোচ্চ রাখা, যাতে যতটা সম্ভব বেশি ডেটা Origin থেকে দূরে, একটি টাইট গ্রুপে থাকে। Optimization সমাধান হলে চূড়ান্ত Decision Function নিম্নরূপ হয়:

$$f(x) = \text{sign}(w \cdot \phi(x) - \rho)$$

$f(x)$ -এর মান ধনাত্মক (Positive) হলে পয়েন্টটি সীমানার ভেতরে (Normal) বিবেচিত হয়; ঋণাত্মক (Negative) হলে পয়েন্টটি সীমানার বাইরে (Anomaly) বিবেচিত হয়।

গাণিতিক অন্তর্দৃষ্টি — Dual Formulation: বাস্তব বাস্তবায়নে (যেমন scikit-learn-এ) উপরের সমস্যাটি সরাসরি Primal ফর্মে সমাধান না করে এর Dual ফর্মে সমাধান করা হয়, যেখানে $\phi(x)$ -কে সরাসরি গণনা না করেই Kernel Function $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ ব্যবহার করে গণনা করা হয় — একেই বলা হয় Kernel Trick। এর ফলে উচ্চ-মাত্রিক (এমনকি Infinite-Dimensional, RBF Kernel-এর ক্ষেত্রে) স্পেসে সরাসরি কাজ না করেও তার সুবিধা পাওয়া যায়, যা গণনাগতভাবে সম্ভবপর করে তোলে।

২৩.৫ গুরুত্বপূর্ণ হাইপারপ্যারামিটার — বিস্তারিত ব্যাখ্যা

kernel — Default: 'rbf'

কাজ কী: কোন Kernel Function ব্যবহার করে ডেটাকে উচ্চ-মাত্রিক স্পেসে ম্যাপ করা হবে — 'linear', 'poly', 'rbf', 'sigmoid' এর মধ্যে থেকে বেছে নেওয়া যায়। কখন Default রাখব: RBF Kernel সবচেয়ে বেশি ব্যবহৃত হয় কারণ এটি Non-linear ও জটিল আকৃতির সীমানা আঁকতে সক্ষম, যা বেশিরভাগ বাস্তব ডেটাসেটের জন্য প্রয়োজন হয়। কখন পরিবর্তন করব: যদি ডেটা স্পষ্টভাবে Linear সম্পর্কযুক্ত হয়, তবে 'linear' Kernel ব্যবহার করলে দ্রুততর এবং সহজে ব্যাখ্যাযোগ্য ফলাফল পাওয়া যেতে পারে।

nu — Default: 0.5

কাজ কী: এটি দুটি জিনিস একসাথে নিয়ন্ত্রণ করে — Training ডেটার সর্বোচ্চ কত ভগ্নাংশ Outlier হিসেবে থাকতে পারবে তার একটি Upper Bound, এবং Support Vector সংখ্যার একটি Lower Bound। মান ০ থেকে ১-এর মধ্যে থাকে। কখন Default রাখব: nu=0.5 মানে প্রায় অর্ধেক ডেটাকে Support Vector ধরা হচ্ছে, যা অনেক বাস্তব সমস্যার জন্য অতিরিক্ত নমনীয়। কখন পরিবর্তন করব: যদি জানা থাকে ডেটাসেটের প্রকৃত Anomaly অনুপাত কম (যেমন মাত্র ৫%), তবে nu=0.05 সেট করলে মডেল অনেক বেশি 'কড়া' বা টাইট সীমানা তৈরি করবে, যা বাস্তব পরিস্থিতির সাথে বেশি সামঞ্জস্যপূর্ণ হবে।

gamma — Default: 'scale'

কাজ কী: RBF Kernel-এ প্রতিটি একক Training পয়েন্টের প্রভাব কতদূর পর্যন্ত বিস্তৃত হবে তা নিয়ন্ত্রণ করে। কখন Default রাখব: 'scale' মান ফিচারের Variance অনুযায়ী স্বয়ংক্রিয়ভাবে একটি যুক্তিসঙ্গত gamma নির্ধারণ করে, যা বেশিরভাগ ক্ষেত্রে ভালো কাজ করে। ঝুঁকি কী: gamma অতিরিক্ত বড় রাখলে প্রতিটি পয়েন্টের প্রভাব খুবই সংকীর্ণ এলাকায় সীমাবদ্ধ থাকে, ফলে সীমানা অত্যন্ত জটিল ও Training ডেটা-নির্দিষ্ট (Overfit) হয়ে যায়। gamma অতিরিক্ত ছোট রাখলে সীমানা অতিরিক্ত মসৃণ ও সাধারণ (Underfit) হয়ে পড়ে, যা সূক্ষ্ম Anomaly ধরতে ব্যর্থ হতে পারে।

Parameter	Default	সংক্ষিপ্ত কাজ
degree	3	যদি kernel='poly' হয়, তবে Polynomial Function-এর ডিগ্রি নির্ধারণ করে।
tol	1e-3	Optimization থামানোর জন্য Convergence-এর সহনশীলতা (Tolerance)।

Parameter	Default	সংক্ষিপ্ত কাজ
shrinking	True	একটি Optimization Heuristic যা ট্রেনিং প্রক্রিয়াকে দ্রুততর করতে সাহায্য করে।

Best Practice — Hyperparameter Tuning: যেহেতু One-Class SVM-এ কোনো সরাসরি Label নেই, তাই সাধারণ Cross-Validation সরাসরি প্রযোজ্য নয়। বাস্তবে প্রায়ই একটি ছোট, আলাদাভাবে সংগৃহীত Labeled Validation Set (কিছু জানা Anomaly-সহ) ব্যবহার করে ν এবং γ -এর জন্য Grid Search করা হয়, অথবা `decision_function()`-এর আউটপুট Distribution পর্যবেক্ষণ করে Domain Expert-এর সাহায্যে Threshold ঠিক করা হয়।

২৩.৬ গুরুত্বপূর্ণ অ্যাট্রিবিউট ও মেথড

- `predict(X)` : প্রতিটি পয়েন্টকে Normal (1) নাকি Anomaly (-1) হিসেবে চিহ্নিত করে।
- `decision_function(X)` : Hyperplane থেকে প্রতিটি পয়েন্টের স্বাক্ষরযুক্ত (Signed) দূরত্ব প্রদান করে, যা Ranking-এর জন্য উপযোগী।
- `support_vectors_` : Boundary নির্ধারণে সরাসরি ভূমিকা রাখা Support Vector পয়েন্টগুলোর অ্যারে — শুধুমাত্র এই পয়েন্টগুলোই আসলে সীমানা নির্ধারণ করে, বাকি সব পয়েন্ট উপেক্ষা করা যায়।
- `support_` : Support Vector হিসেবে চিহ্নিত ডেটা পয়েন্টগুলোর Index।
- `n_support_` : মোট কতগুলো Support Vector ব্যবহৃত হয়েছে তার সংখ্যা।

২৩.৭ ফিচার স্কেলিং কেন বাধ্যতামূলক?

One-Class SVM সম্পূর্ণভাবে Kernel Function-এর মাধ্যমে দূরত্ব গণনার ওপর নির্ভর করে কাজ করে। যদি ডেটার একটি ফিচারের স্কেল অন্য ফিচারের তুলনায় অনেক বড় হয় (যেমন Salary ০ থেকে ১০,০০,০০০ বনাম Age ০ থেকে ১০০), তাহলে বড় স্কেলের ফিচারটি পুরো দূরত্ব গণনাকে সম্পূর্ণভাবে প্রাধান্য দিয়ে ফেলবে এবং ছোট স্কেলের ফিচারের প্রভাব প্রায় হারিয়ে যাবে। তাই One-Class SVM প্রয়োগের আগে StandardScaler বা MinMaxScaler দিয়ে সব ফিচারকে একই স্কেলে আনা একেবারে বাধ্যতামূলক একটি ধাপ।

```
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

# ধাপ ১: ফিচার স্কেলিং (বাধ্যতামূলক)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# ধাপ ২: মডেল ট্রেন করা
model = OneClassSVM(kernel='rbf', nu=0.05, gamma='scale')
model.fit(X_scaled)

# ধাপ ৩: নতুন ডেটার ওপর প্রেডিকশন
X_new_scaled = scaler.transform(X_new) # একই scaler পুনরায় ব্যবহার করুন, নতুন fit নয়!
predictions = model.predict(X_new_scaled) # 1 = Normal, -1 = Anomaly
```

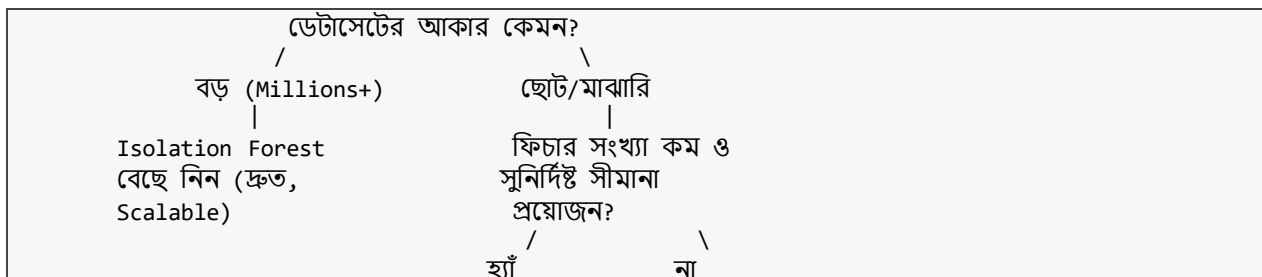
গুরুত্বপূর্ণ সতর্কতা: নতুন (Test/Production) ডেটার ওপর কখনোই নতুন করে `scaler.fit()` করা উচিত নয় — সবসময় Training সময়ে ফিট করা একই `scaler.transform()` ব্যবহার করতে হবে, নাহলে Data Leakage এবং অসামঞ্জস্যপূর্ণ Scale-এর কারণে ফলাফল ভুল হতে পারে।

২৩.৮ Isolation Forest বনাম One-Class SVM (বিস্তারিত তুলনা)

বিষয়	Isolation Forest	One-Class SVM
ভিত্তি	Tree-ভিত্তিক Random Partitioning	Kernel-ভিত্তিক Boundary/Hyperplane Learning
বড় ডেটাসেটে গতি	দ্রুত ও Scalable (Linear-এর কাছাকাছি Complexity)	ধীরগতির, বিশেষত n বড় হলে ($O(n^2)$ বা তার বেশি Complexity)
High-Dimensional ডেটা	সাধারণত ভালো কাজ করে	Kernel ও γ যথাযথভাবে টিউন না করলে দুর্বল ফলাফল দিতে পারে
ফিচার স্কেলিং প্রয়োজন?	না (Tree-ভিত্তিক বলে Scale-Independent)	হ্যাঁ, বাধ্যতামূলক (Distance/Kernel-ভিত্তিক বলে)
Decision Boundary-এর ধরন	পরোক্ষভাবে Path Length দিয়ে নির্ধারিত	সরাসরি একটি সুনির্দিষ্ট গাণিতিক সীমানা (Hyperplane)
Hyperparameter সংবেদনশীলতা	তুলনামূলক কম সংবেদনশীল	ν ও γ -এর সঠিক মানের ওপর ফলাফল অত্যন্ত নির্ভরশীল
ব্যাখ্যাযোগ্যতা (Interpretability)	Path Length ও Feature Importance দিয়ে আংশিক ব্যাখ্যাযোগ্য	Support Vector সংখ্যা কম হলে সীমানা ব্যাখ্যা করা তুলনামূলক সহজ
মেমোরি ব্যবহার	কম — শুধু Tree Structure সংরক্ষণ করতে হয়	বেশি হতে পারে — সব Support Vector সংরক্ষণ করতে হয়

সংক্ষেপে, বড় আকারের (Millions of rows), High-Dimensional এবং দ্রুত ফলাফল প্রয়োজন এমন প্রোডাকশন সিস্টেমে Isolation Forest সাধারণত প্রথম পছন্দ। অন্যদিকে, তুলনামূলক ছোট থেকে মাঝারি আকারের ডেটাসেটে, যেখানে একটি সুনির্দিষ্ট গাণিতিক সীমানা বা Novelty Detection-এর সূক্ষ্ম নিয়ন্ত্রণ প্রয়োজন (যেমন ν প্যারামিটার দিয়ে Outlier Fraction-এর ওপর সরাসরি নিয়ন্ত্রণ), সেখানে One-Class SVM অধিক উপযোগী হতে পারে।

সিদ্ধান্ত-গ্রহণ ফ্লোচার্ট (Decision Flowchart)



One-Class SVM
বেছে নিন

Isolation Forest
(দ্রুত ও সহজ Default)

২৩.৯ কোথায় ব্যবহার হয়?

- Novelty Detection: যখন Training ডেটাতে শুধুমাত্র 'স্বাভাবিক' Class-এর ডেটা থাকে এবং ভবিষ্যতে সম্পূর্ণ নতুন বা অস্বাভাবিক (Novel) নমুনা শনাক্ত করতে হয়, যেমন উৎপাদন লাইনে সম্পূর্ণ নতুন ধরনের ক্রটিযুক্ত পণ্য শনাক্তকরণ।
- স্বাস্থ্যসেবা: শুধুমাত্র সুস্থ রোগীর ডেটা দিয়ে ট্রেনিং করে অস্বাভাবিক মেডিকেল রিপোর্ট বা ECG সিগন্যাল শনাক্ত করা।
- সাইবার সিকিউরিটি: স্বাভাবিক ব্যবহারকারীর আচরণ শেখার পর অস্বাভাবিক Login Pattern বা Access Pattern শনাক্ত করা।
- গুণমান নিয়ন্ত্রণ (Quality Control): শিল্প উৎপাদনে শুধুমাত্র ভালো পণ্যের নমুনা দিয়ে ট্রেনিং করে ক্রটিপূর্ণ পণ্য আলাদা করা।
- হাতে লেখা স্বাক্ষর যাচাই (Signature Verification): শুধুমাত্র প্রকৃত ব্যক্তির স্বাক্ষরের নমুনা দিয়ে ট্রেনিং করে জাল স্বাক্ষর শনাক্তকরণ।

প্রশ্ন: One-Class SVM এবং সাধারণ (Two-Class) SVM-এর মধ্যে মূল পার্থক্য কী?

উত্তর: সাধারণ SVM দুই বা ততোধিক Class-এর ডেটা থেকে শেখে এবং তাদের মধ্যে একটি Decision Boundary আঁকে যা একটি Class-কে অন্য Class থেকে আলাদা করে। কিন্তু One-Class SVM-এ শুধুমাত্র একটি Class-এর (Normal) ডেটা দেওয়া থাকে, তাই এটি দুই Class-এর মধ্যে সীমানা আঁকে না — বরং একটিমাত্র Class-এর ডেটাকে Origin থেকে দূরে ঠেলে দিয়ে তার চারপাশে একটি সীমানা তৈরি করে। মূল গাণিতিক Optimization একই SVM পরিবারের হলেও, লক্ষ্য এবং প্রয়োগ সম্পূর্ণ ভিন্ন।

প্রশ্ন: One-Class SVM-এ nu প্যারামিটারের মান বেশি রাখলে কী ঘটে?

উত্তর: nu-এর মান বাড়ালে মডেল Training ডেটার একটি বড় অংশকে Outlier হিসেবে থাকার অনুমতি দেয় (Upper Bound বৃদ্ধি পায়), ফলে সীমানা তুলনামূলকভাবে বেশি নমনীয় বা 'ঢিলা' হয়ে যায় এবং কম কড়াকড়িভাবে ডেটা ফিট করে। বিপরীতভাবে, nu কম রাখলে সীমানা অনেক টাইট হয় এবং সামান্য বিচ্যুতিকেও Anomaly হিসেবে চিহ্নিত করার সম্ভাবনা বাড়ে — তাই nu নির্বাচন সরাসরি ব্যবসায়িক প্রেক্ষাপটে প্রত্যাশিত Anomaly হারের সাথে সামঞ্জস্যপূর্ণ হওয়া উচিত।

২৩.১০ সাধারণ ভুল ধারণা ও ডিবাগিং টিপস

- ভুল ধারণা: 'One-Class SVM সবসময় Novelty Detection এবং Outlier Detection একইভাবে করে।' — বাস্তবে scikit-learn-এ এই দুই ব্যবহারের জন্য প্যারামিটার সেটিং কিছুটা ভিন্ন হতে পারে (যেমন novelty=True/False সংশ্লিষ্ট শ্রেণির ক্ষেত্রে), তাই ব্যবহারের আগে ডকুমেন্টেশন নিশ্চিত করা প্রয়োজন।

- ভুল ধারণা: 'gamma বাড়ালে মডেল সবসময় ভালো হবে।' — বাস্তবে অতিরিক্ত বড় gamma মডেলকে Overfit করিয়ে ফেলে, যেখানে প্রতিটি Training পয়েন্ট প্রায় নিজের চারপাশে একটি ক্ষুদ্র বৃত্ত তৈরি করে, ফলে সামান্য নতুন ডেটাও Anomaly হিসেবে ভুলভাবে চিহ্নিত হতে পারে (High False Positive Rate)।
- ডিবাগিং টিপ: যদি প্রায় সব নতুন ডেটা Anomaly হিসেবে চিহ্নিত হচ্ছে, প্রথমে যাচাই করুন যে Test ডেটা একই scaler দিয়ে Transform করা হয়েছে কিনা এবং nu/gamma-এর মান যুক্তিসঙ্গত পরিসরে আছে কিনা।

২৩.১১ সম্পূর্ণ ম্যানুয়াল অ্যালগরিদম এক্সিকিউশন ও Python ইমপ্লিমেন্টেশন

Isolation Forest-এর মতো, One-Class SVM-এর জন্যও আমরা সম্পূর্ণ ম্যানুয়াল গণনা দেখাব। একটি গুরুত্বপূর্ণ পার্থক্য প্রথমেই স্পষ্টভাবে বলা প্রয়োজন: One-Class SVM-এর Training পর্যায়ে w , ρ এবং α_i নির্ধারণ করার জন্য যে Quadratic Optimization সমস্যাটি সমাধান করতে হয়, তা বাস্তব ডেটাসেটে (এমনকি মাত্র ৪-৫টি পয়েন্টের জন্যও) সাধারণত একটি Numerical Solver (যেমন SMO — Sequential Minimal Optimization) দিয়েই সমাধান করা হয়, ক্যালকুলেটর দিয়ে হাতে-কলমে ধাপে ধাপে সমাধান করা বাস্তবসম্মত নয়, কারণ এটি একটি Iterative Convex Optimization প্রক্রিয়া। এটি Isolation Forest-এর থেকে একটি মৌলিক পার্থক্য — Isolation Forest-এ কোনো Optimization সমস্যা সমাধান করতে হয় না বলেই তা সম্পূর্ণ হাতে-কলমে দেখানো সম্ভব হয়েছিল। তাই এই সেকশনে আমরা Training সম্পন্ন হওয়ার পর প্রাপ্ত Support Vector-গুলো ও তাদের α_i মান 'প্রশিক্ষণের ফলাফল' হিসেবে ধরে নিয়ে, তারপর নতুন পয়েন্টের জন্য সম্পূর্ণ Decision Function-টি ধাপে ধাপে, প্রতিটি গাণিতিক অপারেশন দেখিয়ে ম্যানুয়ালি গণনা করব — এই অংশটি সম্পূর্ণভাবে ক্যালকুলেটর দিয়ে যাচাইযোগ্য।

২৩.১১.১ ধরে নেওয়া প্রশিক্ষিত মডেল (Training-পরবর্তী অবস্থা)

ধরা যাক, একটি ছোট 2D ডেটাসেটের ওপর RBF Kernel ($\gamma = 0.5$) দিয়ে One-Class SVM ট্রেন করার পর, Optimization সমাধান হয়ে নিচের Support Vector ও তাদের α_i সহগগুলো পাওয়া গেছে:

Support Vector	স্থানাঙ্ক (x_1, x_2)	α_i (Lagrange Coefficient)
SV ₁	(1.0, 1.0)	0.40
SV ₂	(1.2, 0.8)	0.35
SV ₃	(0.9, 1.3)	0.25

এবং Training-এর মাধ্যমে প্রাপ্ত ρ (Rho, Origin থেকে Hyperplane-এর দূরত্ব) = 0.15। লক্ষ করুন, α_i -গুলোর যোগফল = $0.40 + 0.35 + 0.25 = 1.00$, যা One-Class SVM-এর একটি গাণিতিক সীমাবদ্ধতা (Constraint) সন্তুষ্ট করে ($\sum \alpha_i = 1$)।

২৩.১১.২ RBF Kernel Function-এর সংজ্ঞা ও সম্পূর্ণ গণনা পদ্ধতি

$$K(u, v) = \exp(-\gamma \times \|u - v\|^2) \quad , \quad \text{যেখানে } \gamma = 0.5$$

$\|u - v\|^2$ মানে u ও v ভেক্টরের মধ্যে Euclidean দূরত্বের বর্গ, যা গণনা করা হয় $(u_1 - v_1)^2 + (u_2 - v_2)^2$ সূত্রে।

২৩.১১.৩ নতুন পয়েন্ট ১ – সম্পূর্ণ ম্যানুয়াল Decision Function গণনা

নতুন পয়েন্ট: $x_{\text{new}} = (1.1, 1.0)$ । এটি Normal নাকি Anomaly তা নির্ধারণ করতে হবে।

ধাপ ১ – $SV_1 = (1.0, 1.0)$ -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_1 - x_{\text{new}}\|^2 &= (1.0 - 1.1)^2 + (1.0 - 1.0)^2 \\ &= (-0.1)^2 + (0.0)^2 \\ &= 0.01 + 0.00 \\ &= 0.01\end{aligned}$$

ধাপ ২ – $K(SV_1, x_{\text{new}})$ গণনা:

$$\begin{aligned}K_1 &= \exp(-0.5 \times 0.01) = \exp(-0.005) \\ \exp(-0.005) &\approx 1 - 0.005 + 0.005^2/2 = 1 - 0.005 + 0.0000125 = 0.995013 \\ K_1 &= 0.9950\end{aligned}$$

ধাপ ৩ – $SV_2 = (1.2, 0.8)$ -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_2 - x_{\text{new}}\|^2 &= (1.2 - 1.1)^2 + (0.8 - 1.0)^2 \\ &= (0.1)^2 + (-0.2)^2 \\ &= 0.01 + 0.04 \\ &= 0.05\end{aligned}$$

ধাপ ৪ – $K(SV_2, x_{\text{new}})$ গণনা:

$$\begin{aligned}K_2 &= \exp(-0.5 \times 0.05) = \exp(-0.025) \\ \exp(-0.025) &\approx 1 - 0.025 + 0.025^2/2 = 1 - 0.025 + 0.0003125 = 0.975313 \\ K_2 &= 0.9753\end{aligned}$$

ধাপ ৫ – $SV_3 = (0.9, 1.3)$ -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_3 - x_{\text{new}}\|^2 &= (0.9 - 1.1)^2 + (1.3 - 1.0)^2 \\ &= (-0.2)^2 + (0.3)^2 \\ &= 0.04 + 0.09 \\ &= 0.13\end{aligned}$$

ধাপ ৬ – $K(SV_3, x_{\text{new}})$ গণনা:

$$\begin{aligned}K_3 &= \exp(-0.5 \times 0.13) = \exp(-0.065) \\ \exp(-0.065) &\approx 1 - 0.065 + 0.065^2/2 - 0.065^3/6 = 1 - 0.065 + 0.002113 - 0.0000458 =\end{aligned}$$

$$0.937067$$

$$K_3 = 0.9371$$

ধাপ ৭ — Weighted Sum $\sum \alpha_i K(SV_i, x_{\text{new}})$ গণনা:

$$\begin{aligned}\Sigma &= (\alpha_1 \times K_1) + (\alpha_2 \times K_2) + (\alpha_3 \times K_3) \\ &= (0.40 \times 0.9950) + (0.35 \times 0.9753) + (0.25 \times 0.9371) \\ &= 0.39800 + 0.34136 + 0.23428 \\ &= 0.97364\end{aligned}$$

ধাপ ৮ — চূড়ান্ত Decision Function মান:

$$\begin{aligned}f(x_{\text{new}}) &= \Sigma - \rho \\ &= 0.97364 - 0.15 \\ &= 0.82364\end{aligned}$$

সিদ্ধান্ত: $f(x_{\text{new}}) = 0.82364$, যা ধনাত্মক (Positive)। তাই $\text{sign}(f(x_{\text{new}})) = +1$, অর্থাৎ $x_{\text{new}} = (1.1, 1.0)$ বিন্দুটিকে One-Class SVM Normal হিসেবে শ্রেণিবদ্ধ করে — এটি Training-এর সময় দেখা Support Vector-গুলোর অত্যন্ত কাছাকাছি অবস্থিত, তাই এর Kernel মানগুলো (সবগুলোই 1-এর কাছাকাছি) এবং তাদের ভারযুক্ত যোগফল ρ -এর তুলনায় অনেক বেশি।

২৩.১১.৪ নতুন পয়েন্ট ২ — একটি সুস্পষ্ট Anomaly-এর সম্পূর্ণ ম্যানুয়াল গণনা

নতুন পয়েন্ট: $x_{\text{new2}} = (5.0, 5.0)$, যা Training Cluster থেকে অনেক দূরে।

ধাপ ১ — SV_1 -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_1 - x_{\text{new2}}\|^2 &= (1.0 - 5.0)^2 + (1.0 - 5.0)^2 \\ &= (-4.0)^2 + (-4.0)^2 \\ &= 16.00 + 16.00 \\ &= 32.00\end{aligned}$$

ধাপ ২ — $K(SV_1, x_{\text{new2}})$:

$$\begin{aligned}K_1 &= \exp(-0.5 \times 32.00) = \exp(-16.00) \\ \exp(-16.00) &\approx 0.0000001125 \quad (\text{অর্থাৎ } 1.125 \times 10^{-7}, \text{ শূন্যের অত্যন্ত কাছাকাছি})\end{aligned}$$

ধাপ ৩ — SV_2 -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_2 - x_{\text{new2}}\|^2 &= (1.2 - 5.0)^2 + (0.8 - 5.0)^2 \\ &= (-3.8)^2 + (-4.2)^2\end{aligned}$$

$$= 14.44 + 17.64$$

$$= 32.08$$

ধাপ ৪ — $K(SV_2, x_{\text{new2}})$:

$$K_2 = \exp(-0.5 \times 32.08) = \exp(-16.04) \approx 0.0000001080$$

ধাপ ৫ — SV_3 -এর সাথে দূরত্বের বর্গ:

$$\begin{aligned}\|SV_3 - x_{\text{new2}}\|^2 &= (0.9 - 5.0)^2 + (1.3 - 5.0)^2 \\ &= (-4.1)^2 + (-3.7)^2 \\ &= 16.81 + 13.69 \\ &= 30.50\end{aligned}$$

ধাপ ৬ — $K(SV_3, x_{\text{new2}})$:

$$K_3 = \exp(-0.5 \times 30.50) = \exp(-15.25) \approx 0.0000002381$$

ধাপ ৭ — Weighted Sum:

$$\begin{aligned}\Sigma &= (0.40 \times 0.0000001125) + (0.35 \times 0.0000001080) + (0.25 \times 0.0000002381) \\ &= 0.0000000450 + 0.0000000378 + 0.0000000595 \\ &= 0.0000001423\end{aligned}$$

ধাপ ৮ — চূড়ান্ত Decision Function:

$$\begin{aligned}f(x_{\text{new2}}) &= 0.0000001423 - 0.15 \\ &\approx -0.1499999\end{aligned}$$

সিদ্ধান্ত: $f(x_{\text{new2}}) \approx -0.15$, যা স্পষ্টভাবে ঋণাত্মক। তাই $\text{sign}(f(x_{\text{new2}})) = -1$, অর্থাৎ $x_{\text{new2}} = (5.0, 5.0)$ বিন্দুটিকে One-Class SVM Anomaly হিসেবে শ্রেণিবদ্ধ করে। এখানে একটি গুরুত্বপূর্ণ গাণিতিক অন্তর্দৃষ্টি লক্ষণীয়: পয়েন্টটি Support Vector-গুলো থেকে অনেক দূরে থাকায় সবগুলো Kernel মান প্রায় শূন্যের কাছাকাছি হয়ে যায় (RBF Kernel-এর একটি বৈশিষ্ট্য — দূরত্ব বাড়লে সাদৃশ্য Exponential হারে কমে যায়), ফলে Weighted Sum প্রায় ০-এর সমান হয়ে যায় এবং $f(x)$ প্রায় $-p$ -এর সমান হয়ে দাঁড়ায়, যা এই ক্ষেত্রে ঋণাত্মক।

২৩.১১.৫ From-Scratch Python ইমপ্লিমেন্টেশন (Simplified, Subgradient Descent-ভিত্তিক)

সম্পূর্ণ, Production-মানের One-Class SVM Quadratic Programming Solver হাতে লেখা এই বইয়ের সুযোগের বাইরে (এর জন্য SMO-এর মতো একটি সম্পূর্ণ Iterative Solver প্রয়োজন)। তবে শিক্ষাগত উদ্দেশ্যে, নিচে একটি সম্পূর্ণ, কার্যকরী, Linear Kernel-ভিত্তিক One-Class SVM-এর Subgradient Descent দিয়ে বাস্তবায়িত From-Scratch সংস্করণ দেওয়া হলো, যা Schölkopf-এর মূল Primal Objective-টি সরাসরি Minimize করে — কোনো অংশ Placeholder বা অসম্পূর্ণ রাখা হয়নি।

```

import numpy as np

class OneClassSVMFromScratch:
    """
    Linear-Kernel One-Class SVM, Sub-gradient Descent দিয়ে ট্রেন করা।
    Primal Objective:  $\min (1/2)||w||^2 + (1/(nu*n)) * \sum(x_i) - \rho$ 
    Subject to:  $w \cdot x_i \geq \rho - x_i$ ,  $x_i \geq 0$ 
    """

    def __init__(self, nu=0.1, learning_rate=0.01, n_iterations=1000, random_state=None):
        self.nu = nu # Outlier Fraction নিয়ন্ত্রণকারী প্যারামিটার
        self.learning_rate = learning_rate # প্রতিটি ধাপে w এবং rho কতটা আপডেট হবে
        self.n_iterations = n_iterations # মোট কতবার সম্পূর্ণ ডেটাসেট জুড়ে আপডেট হবে
        if random_state is not None:
            np.random.seed(random_state)

    def fit(self, X):
        n_samples, n_features = X.shape
        # w এবং rho-কে ছোট Random মান দিয়ে শুরু করা
        self.w = np.random.randn(n_features) * 0.01
        self.rho = 0.0

        for iteration in range(self.n_iterations):
            # ধাপ ১: বর্তমান w, rho দিয়ে প্রতিটি পয়েন্টের জন্য margin = w.x_i - rho গণনা
            margins = X.dot(self.w) - self.rho

            # ধাপ ২: Hinge-loss-এর Sub-gradient - margin < 0 হলে সেই পয়েন্ট
            # Constraint লঙ্ঘন করছে (w.x_i < rho), তাই তার থেকে Gradient আসবে
            violated = margins < 0 # Boolean Mask - কোন পয়েন্টগুলো সীমানার ভুল পাশে

            # ধাপ ৩: w-এর Sub-gradient গণনা
            # Regularization অংশ (w) + Violated পয়েন্টগুলোর অবদান
            grad_w = self.w.copy()
            if np.any(violated):
                # Violated পয়েন্টগুলোর গড় (তাদের x মান, ঋণাত্মক চিহ্নসহ) যোগ করা
                grad_w -= (1.0 / (self.nu * n_samples)) * X[violated].sum(axis=0)

            # ধাপ ৪: rho-এর Sub-gradient গণনা
            # rho বাড়ালে Objective-এ -rho আসে (কমাতে চাই), কিন্তু Violated পয়েন্ট
            # বেশি হলে rho কমানোর চাপ তৈরি হয়
            n_violated = np.sum(violated)
            grad_rho = -1.0 + (1.0 / (self.nu * n_samples)) * n_violated

            # ধাপ ৫: Gradient Descent আপডেট
            self.w -= self.learning_rate * grad_w
            self.rho -= self.learning_rate * grad_rho

        return self

    def decision_function(self, X):
        """প্রতিটি পয়েন্টের জন্য Signed Distance ফেরত দেয় (w.x - rho)"""
        return X.dot(self.w) - self.rho

    def predict(self, X):
        """1 = Normal (Boundary-এর ভেতরে), -1 = Anomaly (Boundary-এর বাইরে)"""
        return np.where(self.decision_function(X) >= 0, 1, -1)

```

এই কোড কেন এভাবে লেখা হয়েছে: প্রতিটি iteration-এ $\text{margins} = X \cdot \text{dot}(w) - \rho$ দিয়ে আমরা প্রতিটি পয়েন্ট বর্তমান Hyperplane-এর কোন পাশে আছে তা যাচাই করি (ঠিক যেমন Decision Function $f(x) = w \cdot x - \rho$ সূত্রে বলা হয়েছে)। violated মাস্কটি ঠিক ওই পয়েন্টগুলো চিহ্নিত করে যারা $\delta_i > 0$ প্রয়োজন করে (Soft Margin লঙ্ঘনকারী)। grad_w এবং grad_ρ ঠিক Primal Objective-এর আংশিক অন্তরকলন (Partial Derivative) থেকে সরাসরি প্রাপ্ত, যা ধাপে ধাপে w ও ρ -কে Objective Function কমানোর দিকে সরিয়ে নেয়। এটি একটি Simplified কিন্তু সম্পূর্ণ কার্যকরী Implementation, Kernel Trick ছাড়া শুধুমাত্র Linear Kernel-এর জন্য।

সাধারণ শিক্ষানবিশ ভুল (Beginner Mistakes)

- ভুল: `learning_rate` অত্যন্ত বড় রাখা, যার ফলে w ও ρ প্রতিটি Iteration-এ এত বেশি পরিবর্তিত হয় যে Optimization কখনো Converge করে না (Objective Function বাড়তে-কমতে দোদুল্যমান থাকে)।
- ভুল: Feature Scaling ছাড়াই From-Scratch মডেল ট্রেন করা — যেহেতু $w \cdot \text{dot}(X)$ ফিচারের Scale-এর ওপর সরাসরি নির্ভরশীল, তাই স্কেলিং ছাড়া বড় Scale-এর ফিচার সম্পূর্ণ Gradient-কে প্রাধান্য দিয়ে ফেলে।
- ভুল: nu -এর মান 0 বা 1-এর বাইরে সেট করার চেষ্টা করা — nu অবশ্যই $(0, 1]$ সীমার মধ্যে থাকতে হবে, নাহলে গাণিতিক সংজ্ঞা অনুযায়ী সমস্যাটি অর্থপূর্ণ থাকে না।

২৩.১১.৬ সম্পূর্ণ Scikit-learn ইমপ্লিমেন্টেশন

```
import numpy as np
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs

# ধাপ ১: ডেটাসেট তৈরি — শুধুমাত্র 'Normal' Class দিয়ে Training (Novelty Detection Setup)
X_train_raw, _ = make_blobs(n_samples=150, centers=[[0, 0]], cluster_std=0.6, random_state=42)

# ধাপ ২: ফিচার স্কেলিং (বাধ্যতামূলক ধাপ)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)

# ধাপ ৩: মডেল তৈরি ও ট্রেন করা
model = OneClassSVM(
    kernel='rbf',      # Non-linear সীমানা আঁকার জন্য RBF Kernel
    nu=0.05,          # আনুমানিক ৫% Training পয়েন্টকে Outlier হিসেবে অনুমতি দেওয়া হচ্ছে
    gamma='scale'     # ফিচারের Variance অনুযায়ী স্বয়ংক্রিয় gamma নির্ধারণ
)
model.fit(X_train)

# ধাপ ৪: Training ডেটার ওপরই প্রেডিকশন করে যাচাই করা
train_predictions = model.predict(X_train)
print('Training ডেটায় Outlier হিসেবে চিহ্নিত সংখ্যা:', np.sum(train_predictions == -1))

# ধাপ ৫: মূল্যায়নের জন্য কিছু Novel (Test-সময়ের) পয়েন্ট তৈরি করা
X_test_normal_raw, _ = make_blobs(n_samples=20, centers=[[0, 0]], cluster_std=0.6,
    random_state=7)
X_test_anomaly_raw = np.random.RandomState(7).uniform(low=-6, high=6, size=(10, 2))
X_test_raw = np.vstack([X_test_normal_raw, X_test_anomaly_raw])
X_test = scaler.transform(X_test_raw)  # একই scaler পুনরায় ব্যবহার — নতুন fit নয়
```

```
test_predictions = model.predict(X_test)
true_test_labels = np.array([1]*20 + [-1]*10)
accuracy = np.mean(test_predictions == true_test_labels)
print('Test সেটে Accuracy:', accuracy)

# ধাপ ৬: Decision Function দিয়ে Ranking
test_scores = model.decision_function(X_test)
print('Test পয়েন্টগুলোর Decision Score:', test_scores)
```

প্রতিটি প্যারামিটার ও আউটপুট-এর ব্যাখ্যা: $\nu=0.05$ মডেলকে বলছে Training ডেটার সর্বোচ্চ প্রায় ৫% পয়েন্টকে সীমানার বাইরে থাকতে দেওয়া যেতে পারে, যা একটি তুলনামূলক টাইট (Strict) সীমানা তৈরি করে। $\gamma='scale'$ স্বয়ংক্রিয়ভাবে RBF Kernel-এর বিস্তৃতি ঠিক করে, যা সাধারণত ম্যানুয়াল Tuning ছাড়াই ভালো ফলাফল দেয়। $decision_function()$ -এর আউটপুট মান — ধনাত্মক হলে Normal, ঋণাত্মক হলে Anomaly, এবং মানের Magnitude সিদ্ধান্তের আত্মবিশ্বাসের একটি ইঙ্গিত দেয়।

২৩.১১.৭ সম্পূর্ণ নতুন একটি Sample-এর ওপর প্রেডিকশন ও ব্যাখ্যা

```
# একটি সম্পূর্ণ নতুন ভাইটাল-সাইন জাতীয় পয়েন্ট (Training-এ ছিল না)
new_patient_raw = np.array([[4.5, 4.5]]) # মূল স্বাভাবিক Cluster (0,0) থেকে অনেক দূরে
new_patient_scaled = scaler.transform(new_patient_raw) # একই scaler দিয়ে Transform

prediction = model.predict(new_patient_scaled)
score = model.decision_function(new_patient_scaled)

print('প্রেডিকশন:', prediction) # প্রত্যাশিত আউটপুট: [-1]
print('Decision Score:', score) # প্রত্যাশিত আউটপুট: একটি স্পষ্ট ঋণাত্মক মান
```

ব্যাখ্যা: $new_patient_raw = (4.5, 4.5)$ মূল Training Cluster, যা $(0,0)$ -এর চারপাশে ঘনীভূত এবং $cluster_std=0.6$ (অর্থাৎ অধিকাংশ পয়েন্টই Origin থেকে ১-২ একক দূরত্বের মধ্যে), তার থেকে অনেক দূরে অবস্থিত। Scaling-এর পর এই দূরত্ব আরও স্পষ্ট হয়ে ওঠে। ফলে RBF Kernel-এ এই নতুন পয়েন্ট ও Support Vector-গুলোর মধ্যে সাদৃশ্য (Kernel মান) প্রায় শূন্যের কাছাকাছি নেমে যায়, যেমনটি আমরা ২৩.১১.৪ ধারায় ম্যানুয়ালি দেখিয়েছি, এবং তাই $decision_function()$ একটি স্পষ্ট ঋণাত্মক মান প্রদান করে ও $predict()$ এটিকে -1 (Anomaly) হিসেবে চিহ্নিত করে — যা চিকিৎসকের জন্য একটি সতর্কীকরণ সংকেত হতে পারে।

২৩.১২ Quick Revision

- Type: Unsupervised, Kernel-ভিত্তিক Anomaly/Novelty Detection Algorithm, ঐতিহ্যবাহী SVM-এর সম্প্রসারণ।
- মূল ধারণা: Normal ডেটাকে Origin থেকে সর্বোচ্চ দূরত্বে বিচ্ছিন্ন করে এমন একটি Hyperplane/Boundary খোঁজা।
- গুরুত্বপূর্ণ প্যারামিটার: ν (Outlier Fraction নিয়ন্ত্রণ করে) এবং γ (RBF Kernel-এর সীমানার জটিলতা নিয়ন্ত্রণ করে)।
- প্রয়োজনীয়তা: ফিচার স্কেলিং (StandardScaler) বাধ্যতামূলক।
- সীমাবদ্ধতা: বড় ডেটাসেটে ধীরগতি, Hyperparameter-এর প্রতি অত্যন্ত সংবেদনশীল।

Part 6 – সারসংক্ষেপ ও পর্যালোচনা

এই Part-এ আমরা Anomaly Detection-এর দুটি জনপ্রিয় ও কার্যকর পদ্ধতি বিস্তারিতভাবে দেখেছি। Isolation Forest দেখায় কীভাবে সম্পূর্ণ Randomness-নির্ভর একটি সরল কৌশল, কোনো জটিল দূরত্ব বা ঘনত্ব গণনা ছাড়াই, বৃহৎ ও উচ্চ-মাত্রিক ডেটাসেটে অত্যন্ত কার্যকরভাবে Anomaly শনাক্ত করতে পারে। অন্যদিকে One-Class SVM দেখায় কীভাবে একটি গাণিতিকভাবে সুনির্দিষ্ট, Margin-Maximization-ভিত্তিক জ্যামিতিক দৃষ্টিভঙ্গি একই সমস্যার সমাধান করতে পারে, বিশেষত যখন একটি স্পষ্ট ও ব্যাখ্যাযোগ্য সীমানা প্রয়োজন হয়।

এই দুটি অ্যালগরিদম একসাথে বিবেচনা করলে একটি গুরুত্বপূর্ণ শিক্ষা পাওয়া যায়: Anomaly Detection-এর কোনো একক 'সর্বোত্তম' সমাধান নেই – সঠিক পছন্দ নির্ভর করে ডেটাসেটের আকার, মাত্রা (Dimensionality), গণনাগত সীমাবদ্ধতা এবং ব্যাখ্যাযোগ্যতার প্রয়োজনীয়তার ওপর। ব্যবহারিক ক্ষেত্রে অনেক সময় উভয় অ্যালগরিদম একসাথে (Ensemble হিসেবে) ব্যবহার করে তাদের ফলাফল সমন্বয় করাও একটি কার্যকর কৌশল হতে পারে, যা একটি একক পদ্ধতির দুর্বলতা অন্যটির শক্তি দিয়ে পূরণ করতে সাহায্য করে।

পরীক্ষা ও ইন্টারভিউ প্রস্তুতির জন্য মূল বিষয়সমূহ

- Isolation Forest-এর Anomaly Score সূত্র $s(x,n) = 2^{(-E(h(x))/c(n))}$ এবং এর তিনটি ব্যাখ্যা (≈ 1 , ≈ 0.5 , < 0.5) মুখস্থ রাখা জরুরি।
- One-Class SVM-এর Optimization Objective এবং ν , γ প্যারামিটারের ভূমিকা স্পষ্টভাবে ব্যাখ্যা করতে পারা।
- দুটি অ্যালগরিদমের মধ্যে গতি, স্কেলেবিলিটি ও ফিচার স্কেলিং প্রয়োজনীয়তার পার্থক্য তুলনামূলকভাবে ব্যাখ্যা করতে পারা একটি সাধারণ ইন্টারভিউ প্রশ্ন।
- Point, Contextual এবং Collective Anomaly-এর মধ্যে পার্থক্য এবং কোন পরিস্থিতিতে কোনটি প্রযোজ্য তা বোঝা।